

École d'Ingénierie

MEMOIRE DE FIN D'ETUDES

Présenté par :

M. EZZOUKHRY Mouhssine

Pour l'obtention du diplôme de Licence

Filière Informatique appliquée

Option Développement Logiciel

Sujet

Application de gestion des commandes payées & entreposage

Sous la direction de :

M. Youssef TAKOURT: Encadrant Entreprise

M. Marouane SLAITAN : Encadrant Académique

Projet de fin d'études soutenu le **08 juillet** 2026, devant le jury composé de:

M. Marouane SLAITANE : Encadrant Académique

M. Abad : Rapporteur

Référence: 0X ----- 2025-2026

Année Universitaire : 2025 -2026

Remerciements

Je tiens à exprimer ma plus sincère gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réussite de ce stage et à l'aboutissement de ce projet. Ce travail n'aurait pu voir le jour sans leur précieux soutien, leurs conseils éclairés et leur accompagnement constant.

Je souhaite tout d'abord adresser mes remerciements les plus chaleureux à Mr. Marouane SALAITANE, mon encadrant pédagogique à l'Université Mundiapolis, pour sa confiance, son professionnalisme et sa disponibilité tout au long de ce projet. Ses orientations pertinentes et ses retours constructifs ont été d'une aide inestimable dans la conduite de mes travaux.

Un merci tout particulier à mon tuteur en entreprise chez KITEA, M. Youssef TAKOURT, pour m'avoir accordé cette opportunité de stage et pour son encadrement technique rigoureux. Son expertise, ses conseils avisés et son soutien quotidien ont grandement contribué à l'enrichissement de mon expérience professionnelle.

Je tiens également à remercier chaleureusement toute l'équipe du Département Logistique et Système d'Information de KITEA pour leur accueil exceptionnel, leur esprit d'équipe et leur volonté de partage. Les échanges techniques et fonctionnels durant ces semaines ont fait de cette expérience un moment d'apprentissage intense et passionnant.

Mes pensées reconnaissantes vont aussi à l'ensemble du corps professoral de l'Université Mundiapolis pour la qualité de la formation reçue, qui m'a permis d'aborder ce stage avec des bases théoriques solides et une réelle capacité d'adaptation face aux exigences du monde professionnel.

Enfin, je ne saurais terminer sans exprimer toute ma gratitude à ma famille pour son soutien indéfectible, ses encouragements constants et sa patience durant ces mois de travail intense. Un merci spécial à mes amis qui ont su me soutenir et m'épauler tout au long de cette aventure professionnelle.

Ce stage a été une expérience humainement riche et professionnellement formatrice. Je garderai de ces mois passés chez KITEA non seulement des compétences techniques renforcées, mais aussi le souvenir d'une équipe soudée et bienveillante.

Résumé

Dans le cadre de notre projet de stage, l'objectif principal est de développer une application web dédiée à la gestion des commandes payées et au suivi de l'entreposage au sein de l'entreprise **KITEA**. Cette application vise à optimiser l'espace de stockage des entrepôts en automatisant le suivi des délais de récupération des marchandises par les clients après leur paiement.

L'application s'interface avec l'ERP Microsoft Dynamics AX pour importer les commandes, gère de manière autonome le cycle de vie du stockage à travers différentes phases temporelles, et applique automatiquement des frais d'entreposage contractuels de 10% en cas de dépassement des délais. Le système intègre également un moteur de notifications pour envoyer des alertes automatisées et générer des documents juridiques (mises en demeure au format PDF) aux jalons clés (J+30 et J+60). Un tableau de bord décisionnel permet aux agents logistiques de piloter l'état de l'entrepôt et de gérer les cas exceptionnels, les retards ou les annulations.

Ce projet a permis de mettre en place une solution industrielle robuste, facilitant la libération de la surface de stockage et réduisant le manque à gagner financier lié à la gestion manuelle. Conçue avec une architecture moderne associant **React** pour le frontend et **Laravel** pour le backend, l'application garantit une réactivité optimale et une traçabilité totale des flux logistiques post-achat.

Mots-clés : Gestion d'entreposage, Suivi logistique, Frais de stockage, Automatisation, Notifications, KITEA, Application web, Laravel, React.

Abstract :

In the context of our internship project, the main goal is to develop a web application dedicated to managing paid orders and tracking warehouse storage within **KITEA**. This application aims to optimize warehouse space by automating the tracking of pickup deadlines for customer orders after payment has been completed.

The application interfaces with the Microsoft Dynamics AX ERP system to import order data, autonomously manages the storage lifecycle across multiple temporal stages, and automatically applies a contractual 10% storage fee when deadlines are exceeded. The system also features a notification engine to dispatch automated email alerts and generate legal documents (formal notice PDFs) at key milestones (D+30 and D+60). A business intelligence dashboard provides logistics agents with tools to monitor warehouse metrics, handle cancellations, and manage operational exceptions.

This project has led to the implementation of a robust industrial solution, facilitating the release of warehouse capacity and minimizing the financial losses caused by manual tracking. Developed using a modern tech stack combining **React** for the frontend and **Laravel** for the backend, the application ensures high responsiveness and complete traceability of post-purchase logistics.

Keywords: Warehouse Management, Logistics Tracking, Storage Fees, Automation, Notifications, KITEA, Web Application, Laravel, React.

Table des matières

Liste des figures

Figure 1: Logo _____ **Error! Bookmark not defined.**

INTRODUCTION

Dans un contexte économique mondial fortement numérisé, la performance d'une grande entreprise ne dépend plus uniquement de la qualité de ses produits, mais également de l'efficacité et de l'agilité de sa chaîne logistique. Le stockage de marchandises, la gestion des flux et la maîtrise des délais de récupération représentent des enjeux financiers et opérationnels majeurs. C'est dans cette dynamique d'optimisation continue que s'inscrit le présent projet de fin d'études, mené au sein de la Centrale Logistique de la société **KITEA**, leader national de l'ameublement et de la décoration au Maroc.

Lors de son parcours d'achat, le client valide et règle sa commande, qui est ensuite mise à sa disposition au niveau des entrepôts centraux. Cependant, la stagnation prolongée de certaines commandes payées non réclamées engendre des goulets d'étranglement logistiques : saturation de l'espace physique, augmentation des coûts d'entreposage et opacité sur la traçabilité des colis à forte valeur. Face à cette problématique, KITEA a exprimé le besoin de structurer un système rigoureux et automatisé de suivi et de régulation du cycle de vie de stockage.

L'objectif principal de ce projet est donc de concevoir et de développer une application web d'automatisation et de gestion des commandes payées en attente de livraison. Ce système repose sur une logique de traitement temporel stricte divisée en trois phases réglementaires : une période initiale de stockage gratuit, une phase d'alerte et de relance à partir du 30ème jour, et enfin l'application d'une mise en demeure assortie de pénalités financières automatiques de 10% au-delà du 60ème jour, tout en intégrant des mécanismes d'exception pour les cas de force majeure.

Pour répondre à ces exigences techniques et métiers, la solution a été architecturée autour de l'écosystème moderne **Laravel (PHP)** pour la robustesse des processus d'arrière-plan (*Cron Jobs* et file d'attente SMTP) et de la bibliothèque **React (JavaScript)** pour la création d'une interface utilisateur dynamique, sécurisée et fluide dédiée aux agents administratifs.

CHAPITRE 1 : CADRE GÉNÉRAL DU PROJET ET ORGANISME D'ACCUEIL

Introduction du chapitre

Ce premier chapitre a pour objectif de poser les fondations contextuelles et institutionnelles du stage réalisé en immersion au sein de **GROUP KITEA ADMINISTRATION**. Face aux exigences accrues de la transformation numérique, il s'avère indispensable de comprendre l'environnement économique, opérationnel et structurel de l'entreprise qui accueille ce projet. Nous présenterons ainsi l'organisme d'accueil à travers son évolution historique, sa fiche technique officielle, ses domaines d'activités stratégiques et la répartition fonctionnelle de ses départements en pôles opérationnels. Enfin, nous détaillerons l'organisation de la Direction des Systèmes d'Information (DSI) afin de cibler avec exactitude le périmètre technique dans lequel s'inscrit notre mission.

1.1 Présentation de l'organisme d'accueil

1.1.1 Description historique et évolution de la société

GROUP KITEA ADMINISTRATION s'impose aujourd'hui comme un opérateur pionnier et une enseigne-phare sur le marché marocain du meuble, de l'équipement de la maison et de la décoration d'intérieur. Fondée en 1993, l'entreprise s'est construite autour d'un concept à l'époque totalement disruptif pour le paysage de la distribution nationale : introduire et démocratiser le meuble en kit, associant modernité, modularité, design contemporain et immédiateté de disponibilité, le tout soutenu par un excellent rapport qualité/prix.

L'histoire de KITEA se caractérise par une croissance spectaculaire et continue. À ses débuts, la structure ne comptait qu'un unique point de vente modeste de 300m² situé à Casablanca, animé par une équipe de seulement 5 employés. Grâce à une écoute active des mutations sociodémographiques et des besoins de confort des ménages marocains, la marque a su s'ancrer dans le quotidien de la population jusqu'à devenir une véritable « Love-brand » incontournable.

Un tournant stratégique majeur a été amorcé en 2008 avec le déploiement du concept KITEA GÉANT. Ces mégastores de très grande superficie (dépassant les 900m² pour la première implantation historique à Casablanca) ont révolutionné l'expérience client en proposant des parcours d'achat immersifs sous forme d'appartements et d'espaces de vie entièrement scénographiés. Fort de ce succès national, le groupe poursuit aujourd'hui sa feuille de route de croissance à l'échelle continentale. À travers l'ouverture de points de vente en Afrique subsaharienne, KITEA exporte son savoir-faire logistique et sa signature commerciale pour répondre à la diversité des marchés internationaux.

1.1.2 Fiche Technique de l'Entreprise

Le tableau ci-dessous synthétise les données administratives, juridiques et quantitatives qui caractérisent GROUP KITEA ADMINISTRATION :

Caractéristique	Détail
Dénomination Officielle	GROUP KITEA ADMINISTRATION
Date de création	1993

Caractéristique	Détail
Siège social	Zone industrielle Ouled Rahou centre, Sidi Maârouf, lotissement medersa Lot N° 1 Ain Chock, Casablanca 20470.
Direction générale	Othmane Benkirane
Forme Juridique	Société Anonyme (S.A.)
Secteur d'activité	Ameublement, Décoration, Équipement de la Maison et Bureautique.
Présence Géographique	16 villes couvertes au Maroc, 33 points de vente actifs.
Surface globale de vente	Plus de 50000m2 de verrières et de surfaces d'exposition au total.
Effectif global	Plus de 1000 collaborateurs hautement qualifiés.
Implantation Internationale	2 magasins d'envergure opérationnels en Afrique subsaharienne.
Site web officiel	www.kitea.com
Téléphone de contact	0522584109

1.1.3 Domaines d'activités et Univers Produits

Pour maintenir son avantage concurrentiel, KITEA déploie une stratégie d'offre extrêmement dense, s'appuyant sur un catalogue dynamique comptant plus de 16 000 références actives. L'assortiment est rigoureusement segmenté et mis en valeur au sein de 7 univers distincts qui structurent le parcours client dans les points de vente :

- 1. Chambres à coucher :** Collections complètes modulaires adaptées aux adultes, aux adolescents et aux enfants (lits, dressings, chevets).
- 2. Salons et Séjours :** Canapés fixes et convertibles, fauteuils, tables basses, et une adaptation locale forte à travers des gammes de salons marocains revisités.

3. **Espaces Salles à manger** : Ensembles de tables, chaises et bahuts associant esthétique et robustesse.
4. **Mobilier d'extérieur** : Solutions d'aménagement pour les terrasses et jardins (salons d'extérieur, parasols).
5. **Cuisines & Rangement** : Modules fonctionnels optimisés pour l'organisation de la maison.
6. **Décoration & Textiles** : Luminaires, tapis, arts de la table et accessoires pour personnaliser les intérieurs.
7. **Bureautique professionnelle** : Prise en charge intégrale des espaces de travail à travers la marque dédiée KITEA PRO.

Le modèle de réussite de KITEA ne repose pas uniquement sur ses produits, mais également sur un écosystème de services intégrés visant la fidélisation : un service de conseil en aménagement, des garanties contractuelles étendues, un service de livraison et de montage gratuits à domicile, ainsi que des facilités de financement par le biais de formules de crédit gratuit.

1.2 Structure Organisationnelle du Groupe KITEA

GROUP KITEA ADMINISTRATION articule sa gouvernance autour d'une séparation claire et étanche entre les départements de *Front Office* (directement en contact avec le marché et les clients) et les fonctions de *Back Office* (directions supports logées au siège et assurant la continuité industrielle).

1.2.1 Les Pôles de Front Office : L'axe commercial

Le Front Office regroupe les directions stratégiques dont l'objectif premier est de stimuler la croissance du chiffre d'affaires, de piloter l'attractivité de la marque et de conquérir de nouvelles parts de marché :

- **Direction Achats** : Pivot central de la chaîne de valeur, elle gère les relations de négociation, de tarification et de conformité avec plus de 35 fournisseurs de premier plan, nationaux et internationaux. Elle orchestre activement une transition industrielle majeure, signée sous l'égide du ministère de l'Industrie marocain, visant à doubler son taux de sourcing local en le faisant passer de 19% à 38% afin d'encourager la production nationale.
- **Direction Commerciale** : Responsable du réseau physique de distribution, elle encadre les managers nationaux et les responsables de secteur. Elle analyse les indicateurs de performance commerciale (KPIs) des 33 magasins et ajuste les grilles promotionnelles et tarifaires selon les formats d'enseigne.
- **Direction Grands Comptes (KITEA PRO)** : Pôle exclusivement dédié aux relations B2B. Elle conçoit des solutions d'aménagement d'espaces sur-mesure pour des donneurs d'ordres institutionnels, des franchises, des chaînes hôtelières et des ensembles tertiaires, en coordonnant une logistique adaptée aux livraisons de gros volumes.
- **Direction E-commerce** : Moteur du développement omnicanal du groupe, cette équipe réunit les experts en marketing digital, en expérience utilisateur (UX/UI), en gestion de trafic et en gestion de la relation client (CRM). Elle anime la plateforme web marchande et veille à l'optimisation des taux de conversion.

- **Service Marketing & Export** : Il assure l'identité visuelle de la marque en coordonnant le visual merchandising des magasins. Parallèlement, le pôle Export pilote la conformité réglementaire, administrative et douanière des flux de marchandises expédiés vers les filiales internationales (Ghana, Côte d'Ivoire, Sénégal, Kenya).

1.2.2 Les Pôles de Back Office : L'infrastructure de support

Le Back Office réunit les expertises internes indispensables à la pérennité, à l'évaluation des performances, à la sécurité et à la solidité financière du groupe :

- **Direction Logistique (Centrale Logistique)** : Supervise la supply chain globale depuis la plateforme d'entreposage de masse jusqu'à la mise à disposition finale des produits. Ses équipes (planificateurs, gestionnaires de stocks, coordinateurs WMS) absorbent une charge opérationnelle très dense, caractérisée par le traitement moyen de 4 000 commandes quotidiennes.
- **Direction Audit & Contrôle de Gestion** : Organe de surveillance de la performance, ce pôle audite les processus internes, assure le suivi rigoureux des enveloppes budgétaires et veille à la rationalisation des coûts d'exploitation.
- **Direction Administrative & Financière (DAF)** : En charge de la gouvernance monétaire de l'entreprise, elle supervise la comptabilité globale, la gestion de la trésorerie, les relations avec les institutions bancaires, la paie, la conformité légale et le suivi juridique des contrats d'affaires.
- **Départements Technique, Qualité & Expérience Client** : Ils veillent à la maintenance des infrastructures physiques et traitent les réclamations complexes, le service après-vente (SAV) et les résolutions de litiges clients.
- **Direction des Systèmes d'Information (DSI)** : Garant technique de l'ensemble de l'entreprise, la DSI conçoit, sécurise et maintient en condition opérationnelle l'infrastructure logicielle et réseau du groupe (notamment l'ERP Microsoft Dynamics AX, les systèmes de gestion d'entrepôt WMS et les infrastructures réseaux). C'est précisément au cœur de cette direction que mon stage s'est déroulé.

1.3 Présentation de la Direction des Systèmes d'Information (DSI)

1.3.1 Structure fonctionnelle et Organigramme de la DSI

La Direction des Systèmes d'Information (DSI) agit comme un partenaire technologique transverse majeur, alignant les ressources numériques sur les impératifs de rentabilité et d'agilité de GROUP KITEA. Afin d'assurer une couverture complète des besoins applicatifs et matériels, la DSI est structurée de la manière suivante :

- **Directeur des Systèmes d'Information (DSI)**
 - *Pôle Infrastructure, Réseaux & Cybersécurité* (Garant de la connectivité et de la protection des données)
 - *Pôle Support Technique & Parc Informatique* (Assistance de proximité aux utilisateurs)
 - *Pôle Administration ERP (Microsoft Dynamics AX) & WMS* (Maintenance des bases de gestion)
 - **Pôle Data Management Team**

- Responsable du Pôle Data Management : Mr. Youssef Takourt
- Développeur Logiciel Full-Stack (Poste de Stage occupé par l'Étudiant)

1.3.2 Mon positionnement au sein de la DSI

Pendant toute la durée de mon stage de fin d'études, j'ai été directement intégré au sein de la Data Management Team, sous la hiérarchie et la supervision de Mr. Youssef Takourt. Ce positionnement stratégique m'a permis d'interagir directement avec les flux de données métiers et financiers transitant dans le système d'information de KITEA. Travailler au sein de cette équipe m'a donné l'opportunité d'appliquer une démarche d'ingénieur full-stack, en manipulant des architectures de bases de données relationnelles et en concevant des applications capables d'automatiser des processus logistiques lourds.

1.4 Objet du stage et introduction de la mission

Dans le cadre de la modernisation et de l'optimisation des flux de la Centrale Logistique nationale de KITEA basée à Casablanca, la DSI a mis en évidence un besoin critique : concevoir une solution automatisée pour surveiller, tracer et réguler les délais d'entreposage des marchandises volumineuses payées que les clients laissent en souffrance en entrepôt.

Ma mission au sein de la Data Management Team a donc consisté à concevoir et développer une solution logicielle moderne et découplée, s'appuyant sur une interface en React.js pour la supervision et une API REST robuste sous Laravel couplée à une base MySQL pour l'exécution des calculs asynchrones et des actions de relance.

L'analyse approfondie du problème de terrain, l'étude critique des anciens processus sur tableur, la définition fine des besoins utilisateurs ainsi que la modélisation des règles métiers et financiers de KITEA feront l'objet du chapitre suivant dédié au Cahier des Charges.

CHAPITRE 2 : CAHIER DES CHARGES ET SPÉCIFICATIONS FONCTIONNELLES

Introduction du chapitre

Ce deuxième chapitre formalise le cadre fonctionnel, technique et organisationnel du projet de gestion et de contrôle de l'entrepôt pour la Centrale Logistique de KITEA à Sidi Maârouf. Après avoir cerné le contexte institutionnel et macro-économique de l'enseigne au cours du premier chapitre, nous traduisons ici les expressions de besoins métiers fournies par la direction opérationnelle en spécifications logicielles exploitables. Nous détaillerons l'analyse de la problématique, la critique constructive de l'existant, les objectifs fonctionnels de la solution web, la matrice des droits d'accès axée sur le rôle pivot de l'Agent, ainsi que la logique mathématique des règles d'affaires et la modélisation relationnelle MySQL locale qui structurent l'architecture backend.

2.1 Contexte, Problématique Métier et Critique de l'Existant

La Centrale Logistique nationale de KITEA, implantée à Casablanca, centralise l'ensemble des flux d'approvisionnement des trente-trois points de vente du Royaume ainsi que de la plateforme e-commerce. Une fois qu'une commande de mobilier ou de marchandises volumineuses, comme des salons, des literies ou des meubles en kit, est intégralement réglée par le client, elle reste momentanément stockée au sein de l'entrepôt en attendant son retrait physique ou sa livraison finale planifiée.

L'absence d'un outil automatisé et dédié au contrôle du vieillissement de ces stocks engendre plusieurs défaillances majeures pour le groupe :

- **Saturation et surcoût de la surface utile** : Les commandes dormantes s'accumulent et encombrant inutilement la surface utile au sol. Cela pénalise la réception des nouvelles collections et ralentit l'ensemble des flux de rotation physiques associés aux opérations de cross-docking et de put-away.
- **Manque à gagner financier contractuel** : Les pénalités d'entreposage, pourtant prévues par les conditions générales de vente au-delà du délai de garde gratuit, ne sont jamais calculées ni appliquées par manque d'outils de traçabilité, de datation et de suivi individualisé.
- **Insécurité juridique face aux litiges clients** : Il existe une réelle difficulté à isoler de manière factuelle les retards imputables aux clients, tels que l'absence lors de la livraison ou les reports de rendez-vous, de ceux causés par les pannes de la chaîne logistique interne de KITEA. Cela fragilise la position de l'enseigne en cas de réclamation ou de procédure de remboursement.

2.1.1 Étude et critique du système existant

Avant le développement de la solution web actuelle, le suivi de l'âge de stockage des commandes reposait sur un traitement manuel particulièrement lourd. Ce processus archaïque imposait une extraction manuelle quotidienne de fichiers de données brutes, sous forme de fichiers plats au format CSV ou Excel, depuis le système d'information central. Par la suite, les équipes logistiques procédaient à une consolidation artisanale au sein de volumineux tableurs Microsoft Excel à l'aide de macro-commandes fragiles et complexes. La dernière étape consistait à appliquer manuellement des formules chronophages pour évaluer l'ancienneté des stocks et tenter d'identifier les anomalies.

Ce fonctionnement historique présentait des lacunes évidentes. Le taux d'erreur humaine demeurait extrêmement élevé lors des manipulations répétitives de données. De plus, la

lourdeur opérationnelle s'avérait pénalisante pour les équipes de la direction des systèmes d'information et de la logistique. Enfin, l'architecture souffrait d'un manque critique de réactivité. Le reporting n'étant pas instantané, l'enseigne subissait la saturation de son entrepôt sans disposer d'indicateurs prédictifs pour agir de manière proactive auprès des clients.

2.2 Périmètre Fonctionnel et Objectifs Globaux

L'application web développée vient remplacer définitivement ces processus manuels en s'articulant autour de quatre grands axes fonctionnels et applicatifs fondamentaux.

2.2.1 Suivi temporel automatisé du cycle de vieillissement

L'application doit calculer de façon continue le temps de séjour de chaque marchandise et faire transiter automatiquement les dossiers à travers quatre états logistiques. La première phase correspond au stockage gratuit, s'étendant du jour du paiement jusqu'au trentième jour, représentant une période de garde standard sans impact financier. La deuxième phase, déclenchée précisément au trentième jour, constitue la phase horodatée de notification qui modifie l'état de la commande et génère les alertes de relance. La troisième phase est la mise en demeure, couvrant la période du trentième au soixantième jour, agissant comme un outil de pré-contentieux logistique. Enfin, la quatrième phase s'active au-delà du soixantième jour pour confirmer l'application définitive des sanctions financières.

2.2.2 Moteur de calcul financier des pénalités

Le système intègre un module de calcul automatique, transparent et irréversible d'un forfait de dix pour cent sur le montant toutes taxes comprises des commandes en souffrance prolongée. Ce traitement se déclenche rigoureusement au soixantième jour de stockage. L'application consigne chaque transaction financière ainsi générée dans un journal d'audit interne, garantissant une traçabilité totale pour les services comptables de KITEA.

2.2.3 Module de régulation des défaillances internes

Pour équilibrer la relation client, l'application assure l'identification automatique des retards de livraison internes à KITEA supérieurs à sept jours par rapport à la date initialement planifiée. Le système marque alors le dossier au statut critique nommé post-délai. Ce basculement ouvre le droit réglementaire pour le client de demander l'annulation de sa commande et d'obtenir un remboursement intégral sous un délai strict de quinze jours.

2.2.4 Automatisation des documents administratifs et alertes

Le dernier axe fonctionnel concerne l'expédition automatique d'e-mails de relance personnalisés via le protocole SMTP en s'appuyant sur des serveurs de messagerie hautement sécurisés. En parallèle, le backend prend en charge la génération dynamique d'actes officiels de mise en demeure au format PDF, respectant les normes d'impression A4 en mode portrait, prêts pour l'envoi légal ou l'archivage numérique.

2.3 Traçabilité Logique des Exigences Métiers

Pour garantir la conformité du développement par rapport aux attentes de la direction, chaque besoin métier exprime une correspondance directe avec un composant ou un point de terminaison du backend Laravel.

L'exigence relative à l'importation automatisée des flux de commandes payées est prise en charge par la méthode d'importation du contrôleur de commandes, retournant un code de succès d'importation ou une erreur de validation de type de données. Le calcul nocturne de l'âge du

stock et le basculement des phases logistiques sont gérés de manière asynchrone par une commande de console planifiée.

La notification par email à J+30 de l'état de stockage s'appuie sur une classe mailable dédiée envoyée par SMTP, faisant passer le statut de l'entrepôt à l'état notifié. L'application de la pénalité financière de dix pour cent au soixantième jour est opérée par le même automate nocturne, qui effectue une insertion transactionnelle dans la table des frais et bascule la commande en mise en demeure.

La génération et le téléchargement de l'acte officiel de mise en demeure au format PDF sont délégués à une méthode spécifique du contrôleur de commandes utilisant la bibliothèque de rendu HTML vers PDF. Le verrouillage de sécurité en cas de force majeure invoqué par un client est géré par une route de mise à jour qui gèle instantanément les compteurs de jours.

La détection et la validation des annulations pour retard interne supérieur à sept jours exploitent une méthode d'annulation dédiée qui met à jour le statut de livraison et calcule la date limite de remboursement. Enfin, la restitution des indicateurs de saturation en temps réel pour le pilotage de l'entrepôt est assurée par le contrôleur du tableau de bord qui renvoie un payload JSON contenant l'ensemble des métriques opérationnelles.

2.4 Acteurs, Rôles et Droits d'Accès (RBAC)

L'accès aux points de terminaison de l'application est restreint par un contrôle d'accès basé sur les rôles, configuré de manière étanche lors de la phase d'authentification des utilisateurs.

2.4.1 L'Agent Logistique : L'Acteur Opérationnel Unique

L'Agent Logistique représente le cœur opérationnel du système sur le terrain. C'est lui qui interagit avec l'interface graphique développée en React au quotidien depuis son poste de travail situé au sein de la Centrale Logistique. Il gère l'avancement linéaire et traite les exceptions des flux physiques en entrepôt. Ses privilèges applicatifs exclusifs se composent des actions suivantes :

- **Consultation et Suivi** : Accéder au tableau de bord en temps réel pour surveiller l'état de saturation des espaces de stockage et visualiser instantanément les indicateurs clés du vieillissement.
- **Planification de Livraison** : Renseigner, modifier et valider la date prévisionnelle de retrait ou de livraison des meubles dans le système via la variable de date de livraison prévue.
- **Gestion des Exceptions** : Activer ou désactiver le commutateur de Force Majeure sur une fiche commande pour geler temporairement le vieillissement du dossier en cas de justification client validée.
- **Traitement des Annulations** : Valider l'annulation définitive et enclencher la procédure de remboursement sous quinze jours pour les commandes en état de défaillance logistique de l'enseigne.

2.4.2 L'Administrateur : Un Profil Passif de Sécurité

Bien que le système intègre un rôle administrateur au niveau de la couche d'authentification et de la création des comptes utilisateurs, ce profil n'exécute aucune action opérationnelle ou fonctionnelle dans l'application. Toutes les fonctionnalités de gestion, de traitement financier, de contrôle des verrous et de pilotage logistique sont centralisées et déléguées à l'Agent sur son

interface. Le rôle d'administrateur reste une entité technique latente, introduite pour respecter l'extensibilité du schéma d'authentification et de futures évolutions des droits, mais ne possède aucun écran ni privilège d'action métier dédié à ce jour.

2.5 Architecture d'Intégration et Traitement des Données (Interface ERP AX)

Une spécificité technique majeure du projet réside dans sa stratégie d'intégration avec l'infrastructure existante de KITEA S.A.

2.5.1 Approche par REST API Découplée

L'application fonctionne de manière totalement autonome avec sa propre base de données locale MySQL. L'interconnexion directe et synchrone avec l'ERP central Microsoft Dynamics AX n'est pas nativement couplée dans le code de l'application afin d'éviter tout couplage fort, de prévenir les pannes en cascade et de préserver les performances logicielles du serveur.

Pour répondre précisément à la demande de la DSI et du superviseur, la mission consistait à concevoir et exposer une passerelle d'accueil standardisée. C'est l'objectif du point de terminaison API REST implémenté sous la route POST /api/importer-depuis-erp. Ce choix architectural offre un découplage parfait : l'application fournit une interface d'entrée sécurisée et rigoureuse, et le superviseur prendra en charge ultérieurement l'injection des données depuis l'ERP en y raccordant directement ses scripts d'extraction AX.

2.5.2 Logique de Validation et Intégrité Référentielle Stricte

Lorsqu'une requête d'importation frappe l'endpoint, le backend Laravel exécute un validateur strict pour sécuriser la cohérence transactionnelle de la base de données locale MySQL avant toute persistance. Le champ représentant l'identifiant de la commande est obligatoire, de type chaîne de caractères, et doit être unique dans la table des commandes. Cette règle d'unicité empêche toute duplication accidentelle d'importation d'un même flux financier.

Le champ correspondant à l'identifiant du client est obligatoire et son existence est rigoureusement vérifiée par clé étrangère dans la table des clients. Cette contrainte garantit de manière absolue que l'automate d'arrière-plan pourra exécuter la relation liant la commande à l'email du client sans lever d'exception de pointeur nul. La date de paiement doit obligatoirement être soumise au format de date valide. Enfin, le montant toutes taxes comprises de la transaction doit être de type numérique et obligatoirement supérieur ou égal à zéro.

Si les types de données sont incorrects ou si les contraintes d'intégrité échouent, l'API rejette immédiatement la tentative d'insertion en renvoyant un code HTTP 422 Unprocessable Entity contenant le dictionnaire détaillé des violations structurelles. Si toutes les données sont valides, la commande est créée localement en base avec ses statuts par défaut, à savoir une livraison non planifiée, un statut d'entrepôt gratuit, des frais non appliqués, une force majeure désactivée et un statut de remboursement non applicable. Le système confirme alors le succès de l'opération en renvoyant un code HTTP 201 Created.

2.6 Règles de Gestion Métier et Algorithmes Backend

L'analyse du code source backend met en évidence trois règles algorithmiques strictes exécutées de façon synchrone ou asynchrone par le Framework.

2.6.1 Algorithme Nocturne du Cycle de Vie

Chaque nuit, une commande automatique scanne les commandes locales dont le statut n'est pas égal à livré, en incluant explicitement les valeurs nulles. Pour chaque dossier, le système calcule

l'ancienneté du stock en jours en mesurant l'écart entre la date du jour et la date de paiement de la commande.

Si le booléen de force majeure a été activé par l'Agent, l'algorithme ignore immédiatement la commande en passant au dossier suivant, ce qui gèle le calcul du temps et protège le client contre l'application des pénalités.

Pour les dossiers normaux dont l'âge est supérieur ou égal à trente jours mais inférieur à soixante jours et dont le statut d'entrepôt est encore gratuit, le système bascule le statut à l'état notifié, insère un log d'audit dans la table des notifications et expédie l'e-mail SMTP de relance.

Dès que l'âge atteint ou dépasse soixante jours et si les frais n'ont pas encore été appliqués, le statut devient mise en demeure. L'application calcule alors une pénalité forfaitaire et irréversible de dix pour cent sur le montant toutes taxes comprises de la commande. Cette somme est enregistrée de manière immuable dans la table des frais, le drapeau des frais appliqués passe à vrai et l'e-mail de mise en demeure est envoyé au client avec l'acte PDF officiel en pièce jointe.

2.6.2 Algorithme de Planification et Contrôle des Verrous

Lorsqu'un Agent tente de planifier une livraison via la méthode de mise à jour dédiée, le système applique deux verrous de sécurité basés sur l'état courant du dossier. D'une part, si la commande affiche un statut d'entrepôt égal à post-délai ou si elle est enregistrée comme annulée, l'action de planification est immédiatement bloquée et le backend retourne un code HTTP 403 Forbidden. D'autre part, la nouvelle date de livraison prévue soumise par l'Agent doit obligatoirement être supérieure ou égale à la date de paiement d'origine de la commande pour garantir la cohérence chronologique des données en base.

2.6.3 Algorithme de Traitement des Défaillances Logistiques

Le système protège également le consommateur en mesurant les retards internes de préparation de commandes de l'enseigne. Si une commande possède le statut planifié et que la date du jour dépasse la date de livraison prévue de plus de sept jours, le système bascule automatiquement le dossier à l'état post-délai, ce qui déclenche une alerte visuelle rouge sur le tableau de bord de l'Agent.

Si l'Agent valide l'annulation demandée par le client, la commande passe au statut de livraison et d'entrepôt annulé. L'application enregistre alors la date d'annulation exacte et calcule automatiquement la date limite de remboursement à exactement quinze jours supplémentaires par rapport à la date du jour, positionnant le statut de remboursement à l'état en attente.

2.7 Modèle de Données Relationnel et Spécifications des Modèles (MySQL Local)

Pour supporter les traitements transactionnels de l'application et sécuriser les jointures SQL lors des contrôles nocturnes, l'architecture de la base de données locale MySQL s'appuie sur une nomenclature stricte. Les clés primaires et étrangères sont explicitées au sein de l'ORM Eloquent de Laravel pour garantir les propriétés ACID et éviter toute collision d'index.

2.7.1 Le Modèle Client (Client)

La classe Client encapsule les informations de l'acheteur final des produits KITEA. Elle définit explicitement sa clé primaire personnalisée via l'attribut `id_client`. Ce modèle centralise l'identité de l'acheteur à travers trois champs essentiels : son nom complet (`nom_complet`), son adresse électronique (email) indispensable au routage des relances, et son numéro de téléphone (`telephone`). Sur le plan relationnel, ce modèle interagit avec l'entité centrale pour assurer le suivi individualisé du vieillissement des stocks par acheteur.

2.7.2 Le Modèle Commande (Commande)

Pivot central de la base de données, la classe Commande est identifiée par la clé primaire `id_commande`. Ce modèle orchestre à la fois les données d'importation de l'ERP AX et les indicateurs logistiques calculés en local.

Son attribut `$fillable` prend en charge une structure dense : le numéro de commande d'origine, l'identifiant du client acheteur (`client_id`), et l'identifiant de l'opérateur interne (`user_id`). Il stocke également les variables temporelles (date de paiement, de livraison prévue, d'annulation ou limite de remboursement à 15 jours), les statuts métiers (livraison, entrepôt, remboursement), ainsi que les deux verrous stratégiques : le drapeau d'application des frais et le commutateur de force majeure.

La classe Commande structure quatre relations clés fondamentales :

- **Relation User (user) :** Une liaison inverse *BelongsTo* rattachant la commande à l'opérateur interne (l'Agent) responsable du dossier via la clé étrangère `user_id`.
- **Relation Client (client) :** Une liaison inverse *BelongsTo* associant la commande à son acheteur final en connectant la clé étrangère `client_id` à la clé primaire `id_client`.
- **Relation Notification (notification) :** Une liaison *HasMany* permettant de récupérer l'historique des alertes SMTP émises pour cette commande via la clé `commande_id`.
- **Relation Financière (frais) :** Une liaison *HasMany* pointant vers les lignes de pénalités de stockage associées via la clé `commande_id`.

2.7.3 Le Modèle Frais (Frais)

Gouverné par la classe Frais, ce modèle gère la persistance comptable des pénalités financières calculées à J+60. Il possède sa propre clé primaire nommée `id_frais`. Sa structure enregistre de manière immuable le montant de la pénalité (les 10% du montant toutes taxes comprises), la date exacte d'application et la clé étrangère `commande_id`. Il implémente une relation de retour *BelongsTo* vers le modèle Commande, interdisant toute existence de frais orphelins en base de données.

2.7.4 Le Modèle Notification (Notification)

Identifié par la clé primaire `id_notification`, le modèle Notification prend en charge l'historisation légale des communications sortantes. Il enregistre de façon séquentielle le type d'alerte (notification à J+30 ou mise en demeure à J+60), la date d'envoi, le statut de délivrabilité du message, et la clé étrangère `commande_id`. Il est rattaché à son dossier parent par une relation inverse *BelongsTo* vers le modèle Commande.

2.7.5 Le Modèle Utilisateur (User)

La classe User gère l'authentification et les profils des opérateurs internes du système. Utilisant les jetons sécurisés de Laravel Sanctum, ce modèle stocke le nom d'utilisateur, l'adresse email professionnelle, le mot de passe chiffré (masqué lors des sérialisations), et le rôle applicatif qui distingue l'Agent opérationnel sur le terrain de l'Administrateur passif. Il implémente une relation *HasMany* vers le modèle Commande via la clé étrangère `user_id`, permettant d'auditer précisément l'ensemble des actions logistiques menées par chaque utilisateur.

Conclusion du chapitre

Le présent cahier des charges met en évidence l'adéquation parfaite entre les besoins opérationnels de la Centrale Logistique de KITEA et la logique applicative implémentée dans le backend Laravel. En centralisant toutes les actions opérationnelles et de contrôle sur le profil actif de l'Agent et en isolant la base locale MySQL via un endpoint d'importation dédié et hautement validé pour l'ERP AX, l'application garantit une gestion rigoureuse, sécurisée et pérenne. Ce cadre fonctionnel et ces règles d'affaires étant solidement posés, nous pouvons aborder le chapitre suivant consacré à la Conception Technique et à l'Architecture Applicative du projet.

Chapitre 3 : Analyse et Conception

Introduction

Après avoir défini les exigences fonctionnelles et le cadre opérationnel de notre application de gestion d'entrepôt pour KITEA, ce troisième chapitre est consacré à la phase fondamentale de modélisation technique. L'objectif est de traduire le cahier des charges logistique en une architecture logicielle robuste et évolutive. Nous détaillerons d'abord la modélisation conceptuelle de la base de données relationnelle à partir de nos modèles réels de production, en mettant un accent particulier sur les mécanismes de traçabilité et de sécurité. Ensuite, nous introduirons la modélisation comportementale et structurelle à travers l'explication détaillée des diagrammes UML officiels du projet (Cas d'utilisation et Classes). Nous présenterons la conception structurelle de l'interface utilisateur React à travers la description fonctionnelle de ses trois zones métiers principales. Enfin, nous exposerons l'architecture technique d'échange de données qui lie l'ensemble des composants du système.

3.1 Modélisation des Données

La persistance des données de notre système repose sur un modèle relationnel structuré pour traduire fidèlement les règles métiers de la centrale logistique KITEA. Afin de garantir l'intégrité référentielle, la traçabilité complète des actions et l'automatisation des alertes, nous avons découpé notre domaine en MySQL en cinq entités principales. Ces entités, implémentées via l'ORM Eloquent de Laravel, permettent de faire la passerelle entre les données brutes issues de l'ERP AX et l'interface utilisateur finale.

Plusieurs tables ont été définies pour structurer les informations et assurer une gestion optimale ainsi qu'un audit rigoureux au sein de l'application :

- **User** : Cette table gère l'authentification et la traçabilité des opérateurs au sein du système. Chaque utilisateur possède un identifiant unique (id: BigInt), un nom unique (nom_utilisateur: String), une adresse de contact (email: String), un mot de passe chiffré (mot_de_passe: String) ainsi qu'un rôle applicatif (role: Enum(agent, administrateur)) déterminant ses privilèges d'accès.
- **Client** : Cette table regroupe les informations d'identification des destinataires des commandes. Chaque client est caractérisé par un identifiant unique (id_client: BigInt), son nom ou raison sociale (nom_client: String), son adresse électronique (email: String) et son numéro de contact (telephone: String).
- **Commande** : Table pivot de l'application connectée à l'ERP AX. Elle stocke l'identifiant de la commande (id_commande: BigInt), sa date d'acquisition financière (date_paiement: Date), le volume de la transaction (montant_ttc: Decimal), le suivi logistique (statut_livraison: ENUM(non planifiée, planifiée, livrée)) et la planification de sortie de l'entrepôt (date_livraison_prevue: Date). Le suivi des règles de stockage y est matérialisé par statut_entrepot: ENUM(gratuit, notifie, mise_en_demeure, post_delai) et l'indicateur d'application des pénalités frais_appliqués: Boolean. Pour l'arbitrage des litiges, elle embarque la clause de blocage force_majeure: Boolean ainsi que les attributs d'annulation : date_annulation: Date, date_limite_remboursement: Date et statut_remboursement: ENUM(non applicable, en attente, effectue). On y trouve également la clé étrangère de l'agent responsable (user_id: BigInt) et du client (client_id: BigInt).

- **Frais** : Cette table enregistre les lignes de pénalités financières applicables en cas de dépassement des délais de garde contractuels. Elle contient un identifiant unique (id_frais: BigInt), le montant de la pénalité (montant: Double) fixé forfaitairement à 10% du montant de la commande, la date de constatation (date_application: Date) et la clé de liaison (commande_id: BigInt).
- **Notifications** : Destinée à l'archivage légal, cette table stocke l'historique des alertes clients. Elle se compose de l'identifiant unique id_notification: BigInt, du type de document émis (type: String), de la date d'exécution (date_envoi: DateTime), de l'état de réception (statut: String) et de la référence du dossier (commande_id: BigInt)

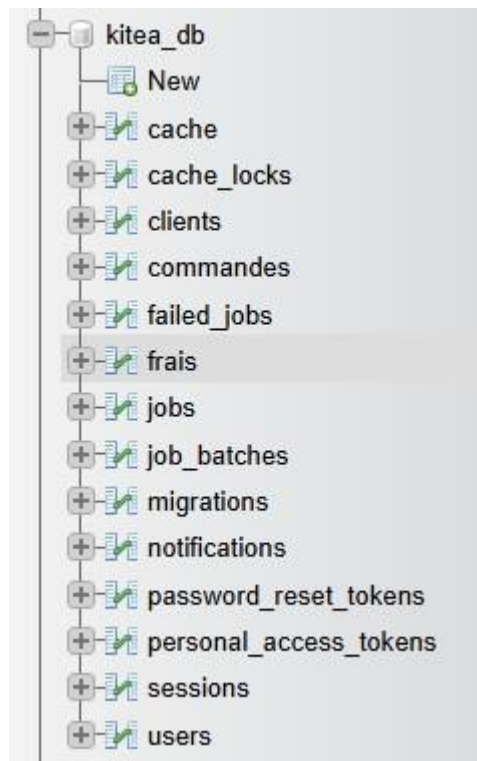


Figure 3.1 : Structure relationnelle des modèles de données KITEA sous MySQL

3.2 Étude des Diagrammes UML

Les diagrammes UML (Unified Modeling Language) fournissent une représentation standardisée et visuelle de l'architecture logicielle de notre application KITEA. Ils formalisent la dynamique des acteurs ainsi que les relations statiques entre les entités du système.

3.2.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation décrit les fonctionnalités du système du point de vue des acteurs externes. Il met en scène trois acteurs distincts interagissant avec la frontière du système KITEA :

- **L'Agent Logistique** : Acteur principal qui interagit directement avec l'interface React. Ses actions métiers exigent toutes une authentification préalable (<<include>> S'Authentifier) :
 - *Consulter le Tableau de Bord*
 - *Programmer une livraison*

- *Forcer l'application des frais*
- *Gérer une annulation de commande (Retard > 7 jours)*
- *Déclarer un cas de force majeure*
- **Le Système ERP AX** : Acteur technique secondaire qui communique en arrière-plan avec l'application pour exécuter le cas d'utilisation métier *Importer les commandes payées*.
- **Le Système Automatisé** : Robot applicatif interne chargé de gérer le temps et d'exécuter de manière transparente les processus asynchrones autonomes :
 - *Calculer les délais de stockage*
 - *Générer une notification automatique (J+30)*
 - *Appliquer les frais d'entreposage (J+60)*
 - *Générer une mise en demeure (J+60)*

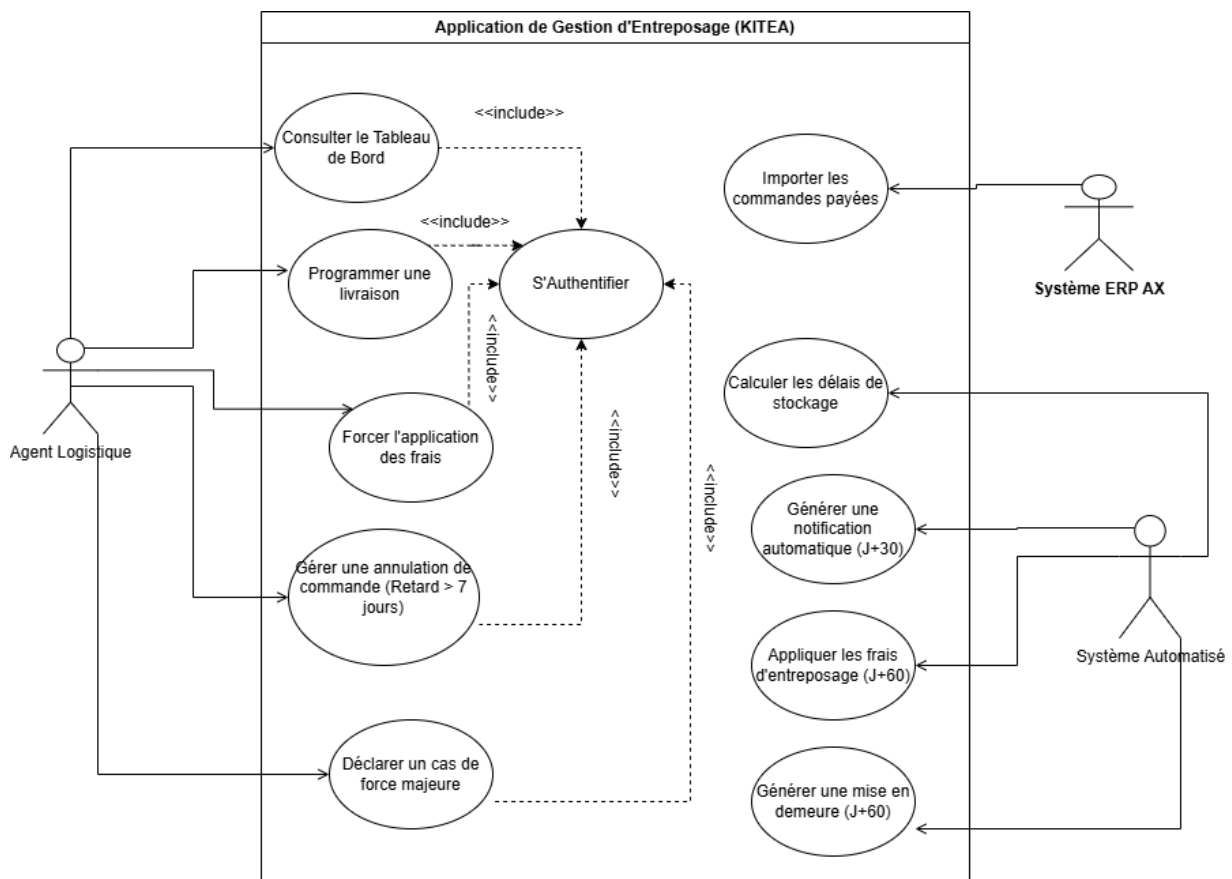


Figure 3.2 : Diagramme de cas d'utilisation - Système de Stockage KITEA

3.2.2 Diagramme de classes

Le diagramme de classes représente la structure statique, logique et le schéma de données objet de notre application de gestion d'entreposage. Contrairement à un simple modèle relationnel de base de données, il met en évidence la correspondance directe entre nos tables physiques MySQL et les classes de notre application backend. Il détaille pour chaque entité ses attributs

fortement typés ainsi que les multiplicités des associations (cardinalités) qui régissent la logique métier du système.

L'analyse des relations de notre modèle met en avant les correspondances suivantes :

- **Relation entre la classe Client et la classe Commande** : Un Client est lié à la classe Commande par une association de type un à plusieurs (1 à plusieurs). Cela signifie qu'un client de la compagnie KITEA peut posséder ou passer plusieurs commandes au fil du temps. À l'inverse, une commande donnée est rattachée de manière exclusive à un seul et unique client. Sur le plan de la persistance, cette relation est matérialisée par la présence de la clé primaire `id_client` au sein de la table Client, laquelle est injectée comme clé étrangère sous le nom `client_id` dans la table Commande.
- **Relation entre la classe User et la classe Commande** : Un User (qui représente l'agent logistique ou l'administrateur connecté au système) gère un ensemble de commandes à travers une association de type un à plusieurs (1 à plusieurs). Cette liaison est fondamentale pour la gouvernance des données : elle permet d'injecter dynamiquement la clé de l'utilisateur (`user_id`) dans l'enregistrement de la commande dès qu'une modification générale ou une action de planification est exécutée. Ce mécanisme garantit une traçabilité totale et répond aux exigences de sécurité en permettant de savoir précisément quel agent a modifié chaque fiche de commande.
- **Relation entre la classe Commande et la classe Notification** : Une Commande déclenche l'émission de zéro à plusieurs (0 à plusieurs) Notifications au cours de son cycle de vie logistique et réglementaire. Une commande nouvellement importée ne possède initialement aucune notification. Si le délai de stockage dépasse les 30 jours gratuits, le système génère une relance simple, puis une mise en demeure automatique à 60 jours. La table Notification intègre ainsi une clé étrangère `commande_id` qui pointe vers la clé primaire `id_commande` de la table Commande.
- **Relation entre la classe Commande et la classe Frais** : Une Commande peut subir zéro à plusieurs (0 à plusieurs) lignes de Frais. Si les marchandises sont retirées pendant la phase gratuite, aucun frais n'est appliqué. En revanche, dès le franchissement du seuil critique des 60 jours, le système applique les pénalités forfaitaires de 10 pour cent de la valeur TTC de la commande. Cette liaison exclusive garantit que chaque ligne de frais calculée reste historiquement et financièrement liée à sa commande d'origine via la clé étrangère `commande_id`.

Cette modélisation orientée objet structure l'ensemble de notre couche d'accès aux données. Elle se traduit directement dans le code backend par l'utilisation de l'ORM Eloquent de Laravel, où les associations sont implémentées via des méthodes natives de type `belongsTo` et `hasMany`, garantissant ainsi l'intégrité référentielle et la fluidité des requêtes.

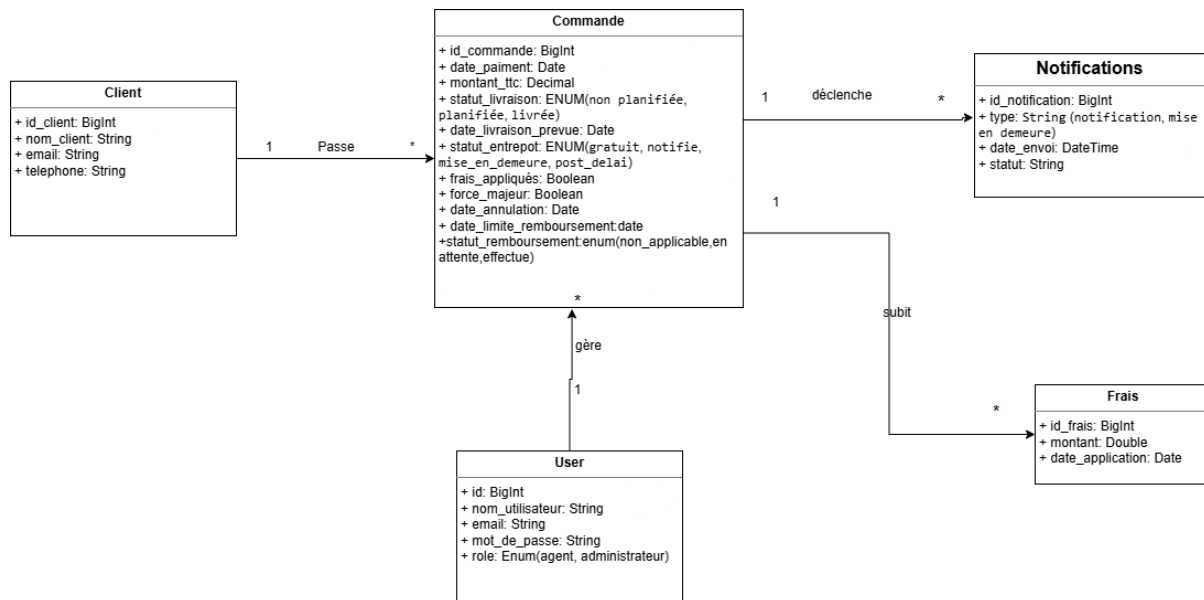


Figure 3.3 : *Diagramme de classes - Modèle de Production Réel*

3.3 Conception de l'Architecture de l'Interface

Afin d'offrir un environnement de travail performant et structuré aux équipes de la centrale logistique, l'interface utilisateur a été découpée conceptuellement en trois grands espaces logiques et modulaires au sein de l'application React. Cette répartition fonctionnelle pose l'agencement des éléments métiers qui seront illustrés en production dans le chapitre 5 :

3.3.1 Zone de Supervision Générale (Composant OrdersGardePage)

Cette zone est pensée pour fournir une vision macroscopique et instantanée de l'état des stocks en entrepôt. Elle orchestre la donnée à travers :

- **Une segmentation temporelle par onglets** : Permettant de basculer d'une catégorie de commandes à l'autre selon l'ancienneté du stockage (Phase Gratuite, Alerte J+30, Litiges J+60).
- **Une table de données dynamique** : Centralisant les statuts logistiques clés (non planifiée, planifiée, livrée) pour chaque référence importée.
- **Un système d'indexation colorimétrique** : Visant à orienter l'œil de l'opérateur vers les dossiers en anomalie (Vert, Orange, Rouge) selon l'urgence du calendrier contractuel.

3.3.2 Zone de Gestion Individuelle (Composant OrderManagementPage)

Ce volet de l'interface est dédié aux actions ciblées sur un dossier unique. Il centralise l'accès aux opérations unitaires requises par l'utilisateur :

- **L'analyse chronologique** : Un panneau récapitulant l'historique complet des événements (date de paiement initial, alertes e-mails déjà transmises par le système).
- **Le module de planification** : Un sélecteur de date dédié à la mise à jour de la date de livraison prévue vers la flotte de transporteurs.
- **L'indicateur d'impact** : Un champ de calcul automatique affichant la valeur théorique de la pénalité de 10% indexée sur la valeur TTC de la marchandise.

3.3.3 Zone d'Arbitrage Réglementaire et Sorties Documentaires

Cet espace regroupe les fonctions de contrôle légal et d'édition de pièces comptables indispensables lors du traitement des litiges :

- **Les déclencheurs d'impression** : Boutons d'accès et d'extraction permettant de télécharger l'Acte PDF de Mise en Demeure ou le Reçu de Pénalité Financière générés par l'API.
- **Le levier de Force Majeure** : Un commutateur d'activation (case à cocher) permettant de geler le traitement automatique d'un dossier en cas d'imprévu contractuel légitime.
- **Le workflow de remboursement pour retard interne** : Une section conditionnelle qui s'active en cas de manquement de l'entreprise (retard supérieur à 7 jours), mettant à disposition de l'opérateur le bouton d'annulation prioritaire pour enclencher le remboursement sous 15 jours.

3.4 Architecture d'Échange des Données

Pour supporter cette conception, l'application adopte un modèle d'architecture découplé de type Client-Serveur via une API REST.

L'interface utilisateur conçue en React (Frontend) fonctionne de manière autonome sur le navigateur de l'utilisateur. Lorsqu'un agent navigue sur le tableau de bord ou modifie une commande, le Frontend émet des requêtes HTTP asynchrones (via des verbes standardisés tels que GET, POST, PUT) vers le serveur Laravel (Backend).

Le serveur Laravel reçoit ces requêtes, valide le jeton d'authentification pour identifier l'agent, traite les règles métiers (comme le calcul des délais de garde ou l'activation du workflow d'annulation), puis effectue les requêtes SQL nécessaires auprès de la base de données MySQL. Une fois l'opération enregistrée, le Backend renvoie une réponse structurée au format JSON vers le Frontend, permettant une mise à jour fluide de l'affichage sans rechargement de la page.

Conclusion

En somme, ce chapitre d'Analyse et de Conception nous a permis de poser des fondations théoriques, logiques et conceptuelles indispensables avant la mise en œuvre technique de la solution. La formalisation des entités en base de données, strictement alignée sur notre diagramme de classes et nos modèles Eloquent réels, garantit une cohérence parfaite avec les flux physiques de stockage de KITEA et les imports de l'ERP AX. L'intégration des rôles utilisateurs et des cas d'utilisation précis répond à un impératif d'automatisation sécurisée et transparente pour la Centrale Logistique, tout en préservant l'exclusivité de la présentation visuelle des espaces pour les chapitres d'implémentation. Forts de cette conception validée et de cette architecture d'échange de données bien définie, nous pouvons désormais aborder le Chapitre 4, dédié à la Réalisation et aux choix technologiques.

Chapitre 4 : Développement et Implémentation

Introduction

Après avoir validé les modèles conceptuels et comportementaux lors de la phase d'analyse, ce quatrième chapitre est consacré à la mise en œuvre technique et à l'implémentation de notre solution de gestion d'entrepôt pour KITEA. Nous détaillerons dans un premier temps les choix technologiques retenus pour le développement de l'application en justifiant la complémentarité des outils de notre stack. Ensuite, nous exposerons l'architecture technique du projet avant de détailler, fichiers à l'appui, l'implémentation de nos fonctionnalités principales.

4.1 Choix technologiques

L'application a été développée en utilisant une combinaison de technologies éprouvées, modernes et largement utilisées, adaptées aux besoins spécifiques du projet ainsi qu'aux contraintes techniques et organisationnelles de l'entreprise KITEA. Le choix de ces technologies repose sur des critères stricts de performance, de compatibilité descendante, de rapidité d'exécution, mais également sur la flexibilité et la sécurité des données logistiques et financières. Voici une présentation détaillée des technologies sélectionnées et de leurs rôles respectifs au sein du projet.

XAMPP :



Figure 4.1 : XAMPP logo

XAMPP est un environnement de développement tout-en-un et multiplateforme qui regroupe plusieurs outils indispensables à la configuration et à la gestion d'un serveur web local. Il permet de simuler un environnement de production complet directement sur une machine locale, facilitant ainsi le cycle de développement, l'écriture du code et les tests d'intégration sans nécessiter de serveur distant dédié pendant les étapes cruciales du projet.

- **Apache** : Un serveur HTTP open-source performant, chargé de traiter les requêtes HTTP entrantes des utilisateurs et d'exécuter l'interpréteur de scripts PHP qui propulse notre backend. Grâce à sa robustesse historique et à sa compatibilité parfaite avec l'architecture de routage, Apache distribue efficacement les flux d'API.
- **MySQL** : Système de gestion de bases de données relationnelles (SGBDR) de référence. Dans ce projet, MySQL assure le stockage, la persistance et la cohérence de l'ensemble des données logistiques (gestion des commandes, calcul des lignes de frais, fichiers clients). Il garantit la sécurité et la fiabilité des opérations financières grâce au support intégral des transactions ACID (Atomicité, Cohérence, Isolation, Durabilité).
- **phpMyAdmin** : Interface web graphique intuitive écrite en PHP, qui permet d'administrer facilement les bases de données MySQL. Cet outil simplifie la création des schémas de tables, le suivi visuel des clés étrangères logistiques, la gestion des index de recherche, et permet d'exécuter des requêtes de test en direct.

Framework Laravel 12 & Eloquent ORM :

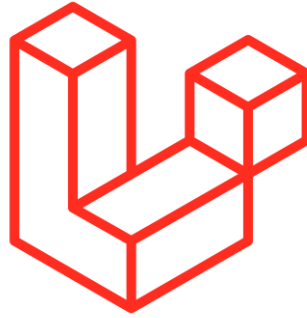


Figure 4.2 : Laravel 12 logo

Laravel 12 représente la toute dernière génération du framework PHP côté serveur. Retenu pour son architecture MVC (Modèle-Vue-Contrôleur) propre, sa sécurité native et sa puissance technique, il sert de moteur central à notre architecture d'API REST. Il intègre l'ORM Eloquent, un système de mappage objet-relationnel avancé.

- **Couche de logique et sécurité** : Laravel 12 gère l'authentification sécurisée des opérateurs de la centrale logistique KITEA par injection de jetons de session, applique des règles de validation strictes sur les formulaires de planification, et orchestre le routage des points d'accès de l'application. Son système de politiques d'accès (Policies) cloisonne les fonctionnalités selon les privilèges des profils Agents ou Administrateurs.
- **Persistance objet avec Eloquent ORM** : Cet outil permet de manipuler les tables de la base MySQL sous forme de classes orientées objet, éliminant totalement l'écriture de requêtes SQL brutes. Eloquent simplifie l'implémentation des relations complexes (comme la liaison un-à-plusieurs entre un client et ses commandes) et prend en charge l'automatisation des processus de traçabilité, notamment lors de l'injection obligatoire du `user_id` de l'agent connecté pour auditer chaque modification de commande.

React & Vite :

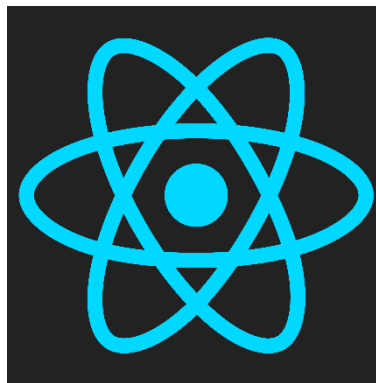


Figure 4.3 : React logo

React est une bibliothèque JavaScript open-source de pointe développée par Meta, spécialement conçue pour concevoir des interfaces utilisateur dynamiques, réactives et hautement modulaires. Couplé au serveur de build de nouvelle génération **Vite**, React permet de bâtir une architecture de type SPA (*Single Page Application*).

- **Composants autonomes et réutilisables** : L'interface utilisateur est entièrement segmentée en blocs logiques indépendants (le composant `OrdersGardePage` pour le tableau de bord ou le composant `OrderManagementPage` pour l'édition de fiches). Cette approche garantit une clarté optimale du code et facilite l'évolution ou la maintenance de l'interface.
- **Expérience utilisateur fluide et instantanée** : Grâce au concept du DOM Virtuel de

React, l'application met à jour uniquement les éléments textuels ou visuels modifiés à l'écran en réponse aux actions de l'agent (comme le basculement entre les onglets temporels de stockage), sans jamais provoquer de rechargement complet de la page du navigateur.

- **Performances accrues avec Vite** : Vite fait office de serveur de développement et d'outil de build ultra-rapide. Il remplace les anciens outils lourds en offrant un démarrage du serveur local en quelques millisecondes et un mécanisme de rechargement à chaud (*Hot Module Replacement*) instantané qui accélère considérablement l'intégration du code Frontend.

Tailwind CSS :



Figure 4.4 : Tailwind CSS logo

Tailwind CSS est un framework CSS orienté "utilitaires" qui révolutionne la mise en page des applications modernes. Contrairement aux frameworks classiques qui imposent des composants graphiques rigides, Tailwind fournit des classes atomiques prêtes à l'emploi qui s'injectent directement au cœur du code JSX de nos composants React.

- **Flexibilité graphique et personnalisation** : Il permet de structurer un design épuré, professionnel et entièrement adapté aux besoins d'affichage de KITEA (tableaux complexes de suivi des commandes, badges d'état, boutons d'action d'arbitrage), tout en éliminant la surcharge de fichiers CSS personnalisés longs à maintenir.
- **Intégration du Responsive Design** : Le framework gère nativement l'adaptation de l'interface aux différentes résolutions d'écran. Les espaces de gestion logistique se repositionnent automatiquement de manière fluide, qu'ils soient consultés sur le poste de travail de bureau d'un gestionnaire ou sur l'écran d'un agent de quai.

Écosystème Frontend & Communication API (Axios) :

Au-delà de la simple présentation de l'interface graphique, l'utilisation de JavaScript (ES6+) a permis d'implémenter une couche de communication réseau robuste, asynchrone et hautement sécurisée avec l'API REST de Laravel 12. Cette architecture découplée repose sur des mécanismes d'échanges de données standardisés :

- **Axios (Client HTTP robuste)** : Cette bibliothèque cliente a été sélectionnée pour interagir avec le backend de manière optimale. Axios sécurise et centralise toutes les requêtes HTTP (GET pour la récupération des flux, POST pour la création des fiches, PUT/PATCH pour la modification des statuts). Elle intègre la sérialisation et la désérialisation automatique des charges utiles (*payloads*) au format JSON, ce qui fluidifie l'intégration des flux logistiques de KITEA entre la base de données et l'affichage. De plus, Axios permet d'injecter des intercepteurs de requêtes afin de joindre automatiquement les en-têtes de sécurité (comme les jetons d'authentification) à chaque appel vers le serveur.
- **Gestion asynchrone et architecture non-bloquante (async/await)** : Afin d'éviter le gel de l'interface utilisateur lors des traitements lourds, la communication s'appuie sur une gestion rigoureuse des promesses JavaScript à travers la syntaxe moderne `async/await`. Ce mécanisme orchestre les flux en arrière-plan : l'agent logistique peut continuer à

naviguer sur l'application pendant que les requêtes de calculs de pénalités financières ou les mises à jour de statuts de commandes s'exécutent. Les réponses du serveur MySQL sont ainsi interceptées et injectées dynamiquement dans l'état (*state*) de React, provoquant un rafraîchissement visuel instantané et transparent pour l'opérateur.

- **Traitement centralisé des erreurs et codes d'état :** La couche réseau implémentée avec JavaScript et Axios intègre un bloc de capture des exceptions (*try/catch*). Ce système analyse les codes d'état HTTP renvoyés par l'API Laravel (par exemple, un code 200 OK pour une validation de commande, un 422 Unprocessable Entity en cas d'erreur de saisie de date, ou un 403 Forbidden pour un refus de privilège). Cela permet de renvoyer des notifications claires et contextualisées à l'écran de l'agent tout en garantissant la stabilité de l'application frontend.

Environnement d'Écriture : Visual Studio Code (VS Code) :



Figure 4.6 : VS Code logo

Visual Studio Code est l'environnement de développement intégré (IDE) principal retenu pour centraliser l'écriture, le refactoring et le débogage de l'ensemble du projet (Frontend et Backend).

- **Écosystème unifié et productivité :** Grâce à une intégration poussée d'extensions spécialisées pour le langage PHP (auto-complétion des classes Laravel, détection d'erreurs) et pour l'écosystème React (JSX, formatage automatique de code via Prettier), VS Code a permis de travailler sur les deux facettes du projet au sein d'une interface unique.
- **Gestion centralisée des terminaux :** Son terminal intégré a facilité l'exécution simultanée et le suivi en temps réel des différents processus d'infrastructure locaux, comme le lancement du serveur backend de l'API Laravel (`php artisan serve`) parallèlement au serveur de build frontend de Vite (`npm run dev`).

Outils de Test et de Débogage (Postman & Chrome DevTools) :



Figure 4.7 : Outils de débogage et d'ingénierie d'API

La mise en place d'un écosystème découplé implique de valider de façon rigoureuse la conformité des échanges de données entre le Frontend et le Backend avant leur liaison définitive.

- **Postman :** Cet outil a été employé pour tester de manière isolée et indépendante chaque point d'accès (Endpoint) développé sous Laravel 12. Il a permis de simuler les requêtes

HTTP envoyées par l'application, de forcer des scénarios d'erreurs pour tester la robustesse du système, et de valider la structure exacte des réponses JSON renvoyées avant de coder leur affichage dans React.

- **Outils de développement Chrome (DevTools)** : Utilisés quotidiennement lors des phases d'intégration visuelle pour inspecter la structure HTML générée, déboguer les scripts d'états à l'aide de la console JavaScript, et suivre minutieusement l'onglet réseau (*Network*) afin de mesurer le temps de réponse de l'API REST et la conformité des données MySQL chargées.

Services de Sortie Documentaire et d'Envoi de Notifications (DomPDF & SMTP Gmail) :

Afin de répondre aux exigences fonctionnelles complexes de la logistique KITEA en matière d'archivage comptable et de communication automatique avec les clients, deux outils d'infrastructure spécifiques ont été configurés :



Figure 4.8 : DomPdf Logo

- **DomPDF (Package Laravel)** : Ce package permet de convertir des vues HTML/CSS directement en documents PDF hautement professionnels et prêts pour l'impression. Au sein de notre architecture backend, DomPDF automatise l'extraction et la mise en page des données MySQL pour générer à la volée les pièces justificatives officielles de l'application, à savoir l'Acte de Mise en Demeure à destination des clients en litige et le Reçu de Pénalité Financière calculé sur la commande.



Figure 4.8 : Gmail Logo

- **Serveur SMTP Gmail (Google Mail Service)** : Pour l'envoi effectif des alertes et des documents, l'application s'appuie sur le protocole de messagerie sécurisé de Gmail. Intégré directement via la configuration des services de messagerie (*Mail Driver*) de Laravel 12, ce choix permet d'envoyer en conditions réelles les e-mails de relance automatique. Cet outil est indispensable pour valider le comportement du robot applicatif : il permet de s'assurer que les alertes de stockage (déclenchées automatiquement au seuil

critique de J+30 et de J+60) sont correctement formatées, qu'elles intègrent les bonnes données dynamiques extraites de MySQL, et qu'elles parviennent instantanément dans la boîte de réception du client.

Système de Contrôle de Version : Git & GitHub :



Figure 4.9 : Git et GitHub logos

Git et GitHub forment le socle de l'infrastructure de gestion de versions et de sauvegarde du code source du projet KITEA.

- **Suivi de versioning avec Git** : Fonctionnant de manière décentralisée en local, Git enregistre l'arborescence chronologique de chaque modification du code. Il a permis de segmenter proprement le développement en isolant les fonctionnalités du Frontend et du Backend au sein de branches de développement distinctes, évitant ainsi tout conflit de code.
- **Centralisation sur GitHub** : La plateforme GitHub sert de dépôt distant sécurisé dans le cloud. Elle garantit une sauvegarde externalisée continue du projet, offre un suivi rigoureux des validations (*Commits*) de code, et assure le maintien d'une base de données de code source propre, documentée et prête pour les phases futures de tests et de déploiement.

4.2 Architecture du projet

L'architecture du projet repose sur une structure bien définie, permettant une gestion claire et évolutive des différentes fonctionnalités de l'application. Voici la description détaillée des fichiers principaux du projet :

4.2.1 Architecture Front-End:

1. **App.jsx** : Le fichier App.jsx constitue le point d'entrée principal et le cœur du routage logique de l'application côté client. Il gère l'état global de l'authentification de l'opérateur logistique à l'aide d'un état React user synchronisé de manière persistante avec le stockage local (localStorage). Au démarrage de l'application, ce fichier exécute un hook d'effet (useEffect) pour valider la présence conjointe d'un jeton de sécurité (auth_token) et des données de session de l'utilisateur. Si un jeton ou un utilisateur est manquant ou invalide, le système procède automatiquement au nettoyage du localStorage afin d'éviter toute corruption d'état, garantissant ainsi qu'aucune session fantôme ne reste active. En plus de l'initialisation de la session, App.jsx encapsule la fonction handleLoginSuccess qui enregistre de façon sécurisée le jeton et le profil utilisateur après une authentification réussie. Il intègre également un sous-composant de sécurité crucial, ProtectedRoute. Ce composant agit comme un intercepteur de routage : il vérifie en temps réel les privilèges d'accès avant d'autoriser l'affichage des interfaces logistiques. Si un utilisateur non authentifié tente de forcer l'accès à une URL interne, ProtectedRoute bloque le rendu et applique une redirection stricte vers la page de connexion (/login). Enfin, App.jsx orchestre l'arbre de navigation global via *React Router*, distribuant les correspondances d'URL vers les modules dédiés comme le tableau de bord, la gestion des dossiers de commande, l'historique des alertes ou le registre de garde.
2. **Layout.jsx** : Le fichier Layout.jsx définit la structure visuelle globale et le gabarit d'organisation de l'espace de travail pour l'ensemble des pages protégées de l'application.

Ce composant joue un rôle d'encapsulation majeur en interceptant le profil de l'utilisateur connecté (`user`) ainsi que la fonction de déconnexion (`onLogout`) pour les transmettre proprement aux éléments enfants de l'interface, notamment au menu de navigation latéral (`Sidebar`). Il configure un conteneur principal exploitant les classes utilitaires de Flexbox pour occuper l'intégralité de l'écran visible tout en neutralisant les barres de défilement globales disgracieuses.

En plus d'intégrer de manière fixe le composant `Sidebar` sur la partie gauche pour assurer une navigation fluide entre les différents registres logistiques, `Layout.jsx` délimite une zone principale de contenu (`<main>`) adaptative et indépendante. C'est à l'intérieur de cet espace que se déploie le composant `<Outlet />` de *React Router*, agissant comme un point d'injection dynamique qui charge automatiquement la vue demandée (comme le tableau de bord ou la gestion d'une commande spécifique) sans recharger la structure environnante. Ce fichier est donc essentiel pour garantir l'homogénéité visuelle de l'application tout en optimisant le flux de données descendant vers les fonctionnalités internes.

3. **Sidebar.jsx** : Le fichier `Sidebar.jsx` gère l'interface de navigation latérale gauche fixe et interactive de l'application, assurant un accès rapide aux différents registres de la centrale logistique. Il s'appuie sur le hook `useLocation` de *React Router* pour détecter en temps réel l'URL courante afin d'appliquer un style visuel distinctif et personnalisé (fond rouge clair et texte bordeaux aux couleurs de l'enseigne) sur l'onglet actif, offrant ainsi un repère visuel immédiat à l'utilisateur. Le fichier intègre au sommet l'affichage dynamique du logo de l'entreprise avec un mécanisme de secours ingénieux (`onError`) : si l'image physique ne parvient pas à se charger, le code intercepte l'erreur pour injecter dynamiquement une alternative textuelle stylisée, évitant toute rupture graphique.

Au-delà de la navigation principale constituée par le tableau de bord, le registre des commandes en garde et l'historique des alertes, `Sidebar.jsx` joue un rôle majeur dans le suivi de la session de l'opérateur en pied de page. Il extrait et affiche les attributs d'identité du profil connecté, comme le nom complet de l'agent administratif et son rôle métier au sein du système. Enfin, il intègre le bouton d'interruption de session via la fonction `handleLogoutClick`. Lors d'un clic, cette action déclenche l'exécution sécurisée du callback `onLogout` passé par le parent pour nettoyer les jetons du stockage local, puis utilise le hook `useNavigate` pour rediriger de force l'utilisateur vers la page d'authentification (`/Login`) en remplaçant l'historique pour interdire tout retour en arrière non autorisé.

4. **Dashboard.jsx** : Le fichier `Dashboard.jsx` fait office de tableau de bord principal et de console d'administration pour la supervision centralisée des stocks logistiques. Ce composant orchestre le chargement asynchrone des indicateurs de performance et du registre global des commandes à travers un hook `useEffect`. Il effectue des requêtes simultanées vers l'API Laravel (`/dashboard/stats` et `/commandes`) via un mécanisme `Promise.all`. Ce fichier intègre des dispositifs de traitement sécurisés pour analyser les réponses, alimenter l'état local des statistiques (`metrics`) et stocker de manière résiliente la liste des transactions sous forme de tableau. En cas de coupure de liaison avec le serveur, il intercepte l'anomalie pour afficher un message d'erreur explicite signalant un problème de synchronisation avec la base de données MySQL.

Au-delà de la collecte de données, `Dashboard.jsx` intègre un moteur de recherche dynamique qui filtre en temps réel les commandes affichées en comparant la saisie de l'opérateur (`searchTerm`) à l'identifiant de la transaction ou au nom complet du client.

L'interface propose une bannière d'alerte critique si des retards de distribution imputables à l'enseigne dépassent le seuil de sept jours, ouvrant un droit de remboursement au profit de l'acheteur. Enfin, le fichier structure visuellement l'affichage sous forme de cartes d'indicateurs clés, ventilant l'occupation des entrepôts selon le cycle de vie légal : le volume global stocké, les relances à trente jours (`phase_notifie_count`), les dossiers en litige et pénalités actives à soixante jours (`phase_mise_demeure_count`), ainsi que le cumul comptable des frais logistiques générés en Dirhams (MAD). Si le chargement est en cours, un indicateur visuel animé est activé, sinon le flux nettoyé est transmis au composant d'affichage `OrderTable`.

5. **`OrderTable.jsx`** : Le fichier `OrderTable.jsx` est un composant d'affichage réutilisable spécialisé dans la modélisation sous forme de tableau des dossiers de commandes en entrepôt. Ce fichier intègre un objet de configuration statique complexe baptisé `STATUS_CONFIG` qui associe à chaque état logistique (`gratuit`, `notifie`, `mise_en_demeure`, `force_majeure`, `annulée`) un ensemble de classes CSS Tailwind spécifiques (couleurs de fond, de bordure et de badges), assurant une différenciation visuelle immédiate des niveaux de criticité. Il intègre également une fonction utilitaire de sécurisation des données nommée `getClientName` qui analyse de manière adaptative l'attribut `client` ; cette méthode est capable de traiter les données qu'elles soient transmises sous forme d'objet natif ou encapsulées sous forme de chaîne JSON, évitant ainsi tout plantage de l'interface en cas d'incohérence du flux de données.

En plus de la mise en forme graphique, `OrderTable.jsx` intègre une barre d'onglets de filtrage dynamique qui permet à l'opérateur de segmenter instantanément les enregistrements par statut d'entrepôt, mettant automatiquement à jour des compteurs numériques pour chaque catégorie. Cette logique est couplée à un système de pagination interne strict limité à cinq éléments par vue (`itemsPerPage`). Le fichier effectue les calculs mathématiques d'indexation (`indexOfFirstItem` et `indexOfLastItem`) et applique un découpage de tableau (`slice`) basé uniquement sur la liste filtrée. Il réinitialise automatiquement l'état de la page courante à un dès qu'un nouveau filtre est appliqué. Enfin, le composant génère un pied de page adaptatif affichant dynamiquement les indices de progression (ex: de 1 à 5 sur 12 dossiers) et fournit des contrôles de navigation réactifs (boutons précédents, suivants et numérotés) qui se désactivent automatiquement lorsque les limites de données sont atteintes.

6. **`OrderManagementPage.jsx`** : Le fichier `OrderManagementPage.jsx` constitue le noyau décisionnel et opérationnel de l'application, dédié à la gestion unitaire, approfondie et réglementaire d'un dossier de commande spécifique. S'appuyant sur le hook `useParams` pour capturer l'identifiant unique (`id_commande`) directement depuis l'URL de navigation, ce composant orchestre la récupération asynchrone des métadonnées associées auprès de la base de données centrale via une requête `api.get` sur l'end-point `/commandes`. Il intègre un mécanisme robuste de filtrage et de gestion d'états d'erreur pour intercepter les anomalies de réseau ou signaler l'absence d'un dossier dans le système central MySQL. Lors du chargement initial, le fichier calcule en temps réel la durée effective de stockage de la commande en comparant l'horodatage d'acquisition de l'ERP AX (`date_paiement`) avec la date du jour, tout en initialisant les formulaires avec la date prévisionnelle de distribution enregistrée.

Sur le plan fonctionnel, ce module centralise les actions métiers stratégiques de l'entrepôt logistique à travers quatre leviers asynchrones majeurs :

- **Planification d'expédition** : Via la méthode `handleScheduleDelivery`, l'opérateur soumet une mise à jour de la date prévisionnelle de sortie (`date_livraison_prevue`) qui bascule dynamiquement l'état d'acheminement du colis.
 - **Activation de clause d'exception** : La fonction `handleToggleForceMajeure` permet de basculer l'état de la clause de "Force Majeure" par une requête POST. Cette action gèle instantanément l'accumulation automatique des frais financiers et des pénalités contractuelles de retard.
 - **Génération d'actes juridiques** : L'action `handleDownloadPDF` initie le téléchargement d'un acte officiel de mise en demeure au format PDF. Le fichier utilise une configuration de réponse de type *Blob* pour traiter le flux binaire brut renvoyé par Laravel, créant un lien de téléchargement éphémère inséré dynamiquement dans le DOM, tout en restreignant strictement cette action aux dossiers ayant atteint le stade légal requis.
 - **Rupture contractuelle et remboursement** : Enfin, le dispositif `handleCancelOrder` intercepte les retards logistiques de l'enseigne supérieurs à sept jours. Après validation d'une boîte de dialogue de confirmation certifiant la réception de la lettre recommandée avec accusé de réception (LRAR) du client, il force l'annulation de la transaction et amorce la procédure de remboursement automatique en stricte conformité avec les règles d'affaires de la centrale.
7. **OrdersGardePage.jsx** : Le fichier `OrdersGardePage.jsx` est un composant de page orienté tableau de bord, configuré pour assurer la supervision macroscopique en temps réel de l'occupation physique et de l'état réglementaire du stock de marchandises au sein des zones de stockage centralisées. Dès le montage du composant, le hook `useEffect` déclenche un appel asynchrone via la méthode `api.get` sur l'end-point `/commandes` afin d'extraire la collection complète des dossiers logistiques, tout en gérant de manière transparente un état local de chargement graphique (`loading`). Pour pallier les risques d'incohérences de chaînes de caractères lors des opérations d'affichage, le fichier se dote d'un dictionnaire de correspondance statique baptisé `tabLabels`, sécurisant le mappage des en-têtes d'onglets de navigation.

L'architecture logicielle de ce module repose sur deux moteurs algorithmiques et logiques clés :

- **Moteur de calcul de vétusté** : La fonction utilitaire `calculateDaysInStorage` prend en charge la conversion de l'horodatage de paiement de l'ERP (`date_paiement`). Elle calcule de manière absolue la différence temporelle en millisecondes par rapport à l'instant présent (`new Date()`) et applique une division mathématique par le facteur temporel d'une journée ($1000 * 60 * 60 * 24$), convertie via `Math.ceil` pour déterminer avec exactitude le nombre de jours écoulés en garde physique.
- **Moteur de segmentation dynamique** : Le filtrage de la collection s'exécute au travers de la constante `filteredOrders`, qui traite l'état d'affichage sélectionné (`activeTab`). Ce filtre regroupe de manière conditionnelle les dossiers selon des critères stricts (ex: l'onglet de criticité litige rassemble via une disjonction logique les commandes sous le statut d'entrepôt mise_en_demeure ainsi que celles bénéficiant d'une clause d'exception `force_majeure`).

Enfin, l'interface graphique génère dynamiquement pour chaque ligne de commande un badge d'état stylisé sous Tailwind CSS. Ce dispositif associe des codes couleurs contextuels (vert pour la phase gratuite, orange pour l'alerte J+30, rouge pour le stade légal et violet pour le gel réglementaire) et intègre un indicateur à puce visuelle, offrant

aux gestionnaires de la plateforme une cartographie instantanée des niveaux d'infraction contractuelle et de la valeur marchande du stock de l'enseigne.

8. **AlertsHistoryPage.jsx** : Le fichier AlertsHistoryPage.jsx fait office de registre de traçabilité et de piste d'audit légale au sein de la plateforme logistique. Sa mission principale est de restituer l'historique complet et immuable des notifications, alertes de stockage (J+30) et actes de mise en demeure (J+60) expédiés aux clients. Dès l'initialisation du composant, le hook useEffect orchestre une requête asynchrone GET vers l'end-point /dashboard/alerts-history. Le cycle de vie de la requête est sécurisé par des états locaux dédiés à la gestion du chargement (loading) et à l'interception des anomalies réseau ou applicatives (error), assurant une isolation stricte des défaillances.

Afin de préserver les performances de l'interface face à des volumes d'archivage potentiellement massifs, ce module intègre un moteur de pagination côté client structuré autour de règles logiques précises :

- **Calcul de segmentation temporelle** : À partir d'un nombre fixe d'éléments par page (itemsPerPage = 5), le composant calcule dynamiquement le volume total de pages via l'expression `Math.ceil(logs.length / itemsPerPage)`.
- **Découpage de la collection** : L'extraction de la sous-section de données à afficher (currentLogs) s'opère par l'usage combiné d'indices de délimitation (indexOfFirstItem et indexOfLastItem) injectés dans la méthode native slice().

Sur le plan de l'affichage et de la conformité métier, le fichier convertit les horodatages système bruts en formats locaux lisibles via la méthode `toLocaleString('fr-FR')`. De plus, il intègre une fonction anonyme auto-exécutée pour harmoniser le rendu du statut de distribution. Ce sous-système isole et normalise les variations de chaînes de caractères renvoyées par l'API (telles que 'sent', 'succès' ou 'delivered') afin d'appliquer dynamiquement des classes de style conditionnelles sous Tailwind CSS, garantissant une lisibilité immédiate des succès ou des échecs de notification.

9. **Login.jsx** : Le fichier Login.jsx constitue le point d'entrée sécurisé et la passerelle d'authentification unique (SSO) de l'application de la centrale logistique. Sa responsabilité principale est de verrouiller l'accès aux tableaux de bord métier en authentifiant les opérateurs via le serveur central de l'enseigne. Lors de la soumission du formulaire, la fonction asynchrone `handleSubmit` intercepte l'événement natif du navigateur via `e.preventDefault()`, réinitialise l'état des erreurs graphiques et bascule l'indicateur de chargement (loading) à true pour neutraliser temporairement le bouton d'action et éviter les requêtes redondantes.

Le mécanisme d'authentification et de gestion de session s'articule autour des processus techniques suivants :

- **Persistance de Session et Hydratation du Contexte** : En cas de réponse favorable de l'API REST (présence confirmée d'un jeton token), le composant extrait la charge utile de l'utilisateur (userPayload) contenant son identifiant électronique, son nom d'usage (nom_utilisateur) ainsi que son habilitation de sécurité (role). Ces données, associées au jeton de sécurité, sont sérialisées en chaînes JSON et inscrites dans le stockage local de l'application (`localStorage.setItem`) avant de propager l'état global via la méthode `onLoginSuccess`.

- **Interception et Typage des Anomalies de Connexion** : La structure try/catch isole les défaillances réseau. Si le serveur renvoie un code de statut HTTP 401 (Unauthorized), le composant intercepte l'anomalie pour notifier l'opérateur d'un rejet des identifiants ou d'un défaut de synchronisation avec l'Active Directory (AD) de l'entreprise. Tout autre code d'erreur déclenche un message d'indisponibilité de la passerelle d'authentification centrale.

L'interface utilisateur s'appuie sur une mise en page adaptative (Responsive Layout) divisée en deux sections distinctes sous Tailwind CSS. La partie gauche héberge le formulaire métier, intégrant des contrôles d'état stricts sur les champs de saisie (email et password), tandis que la partie droite affiche l'identité visuelle de la marque et le contexte fonctionnel du terminal logistique dédié à la gestion des stocks et à la résolution des litiges.

4.2.2 Architecture Back-End :

1. **routes/api.php (Architecture et Cartographie des End-points)** : Le fichier routes/api.php constitue la table de routage centrale et le point d'aiguillage de l'ensemble des requêtes HTTP entrantes adressées au Back-End de l'application logistique. Écrit en PHP sous l'environnement Laravel, ce script orchestre la mise en correspondance (mapping) entre les ressources d'URL sollicitées par le Front-End en React et les méthodes opérationnelles des contrôleurs métiers. Son implémentation sépare rigoureusement la politique d'accès public des périmètres applicatifs protégés.

L'organisation de cette infrastructure réseau repose sur deux couches d'exécution majeures :

- **Passerelle Publique d'Authentification** : La route POST /login est configurée en dehors de tout mécanisme de restriction de session. Elle fait office de point de contact unique pour l'analyse des identifiants et la vérification des profils d'opérateurs auprès du service AuthController.
 - **Périmètre d'Inviolabilité des Données (Garde de Sécurité)** : L'ensemble des fonctionnalités logistiques critiques est encapsulé au sein d'un groupe de routes supervisé par le middleware d'authentification auth:sanctum. Ce garde-barrière intercepte chaque transaction réseau pour valider la présence et l'intégrité du jeton Bearer fourni par le client. Ce groupe sécurise l'accès aux registres d'extraction (GET /commandes, GET /dashboard/stats), aux processus de mutation et planification logistique (PUT /commandes/{id}/planifier-livraison), ainsi qu'aux exécutions d'actes contractuels complexes, tels que le gel des pénalités (POST /toggle-force-majeure), la rupture unilatérale (POST /annuler) ou la génération de flux d'audit (GET /pdf-mise-en-demeure).
2. **CheckStorageDelays.php (Moteur d'Automatisation Asynchrone et Tâches Plannifiées)** : Le fichier CheckStorageDelays.php matérialise le cœur décisionnel autonome et le moteur d'automatisation asynchrone du Back-End. Développé sous la forme d'une commande de console Laravel personnalisée (app:check-storage-delays), ce script est conçu pour s'exécuter chaque nuit de manière transparente. Sa responsabilité est d'analyser l'obsolescence (aging) des marchandises stockées, d'appliquer les pénalités financières contractuelles et d'identifier les retards de livraison imputables à l'enseigne KITEA.

L'architecture logique du processus séquentiel s'articule autour de trois seuils de tolérance et fenêtres réglementaires stricts :

- **Couche d'Abstraction et Analyse d'Éligibilité** : Le script extrait l'ensemble des dossiers dont le statut de distribution n'est pas finalisé (`statut_livraison != 'livrée'`), tout en incluant explicitement les enregistrements non renseignés (`orWhereNull`). Une liaison prédictive (`with('client')`) précharge les données de contact pour optimiser les requêtes SQL. Dès l'ouverture de la boucle de traitement, le script évalue la présence d'un commutateur de blocage (`force_majeure`). Si cette clause d'exception est active, le dossier est immédiatement gelé et extrait du cycle de pénalisation.
 - **Seuil d'Alerte et Notification Commerciale (J+30)** : En mesurant la différence absolue en jours entre l'horodatage d'acquisition de l'ERP (`date_paiement`) et l'instant présent via la bibliothèque Carbon, le script identifie les marchandises stockées depuis plus de 30 jours. Si le statut de l'entrepôt est encore à l'état gratuit, le système fait basculer l'état à notifié, génère une empreinte d'audit immuable dans la table des notifications (`type => 'notification'`) et déclenche l'envoi automatique d'un e-mail de relance au client via la classe `RelanceStockage`.
 - **Seuil Légal et Liquidation des Pénalités (J+60)** : Dès que la rétention physique des colis franchit la barrière critique des 60 jours, le moteur applique une sanction financière automatique. L'état du dossier bascule en `mise_en_demeure` et le script calcule de manière stricte une pénalité forfaitaire correspondant à 10 % du montant total de la commande (`montant_ttc * 0.10`). Cette transaction financière est immédiatement consignée dans le grand livre des frais logistiques (Frais), parallèlement à la notification d'un acte juridique de mise en demeure expédié par voie électronique.
 - **Surveillance des Délais de Distribution Interne (Post-Délai)** : Enfin, le script audite la responsabilité logistique de KITEA. Si une livraison est planifiée mais que la date prévisionnelle de sortie (`date_livraison_prevue`) est dépassée de plus de 7 jours par rapport à la date courante, le système bascule le statut de stockage en `post_delai`. Ce changement d'état lève visuellement une alerte critique sur le tableau de bord de l'opérateur, ouvrant réglementairement le droit à l'annulation et au remboursement intégral au profit du client.
3. **CommandeController.php (Noyau Décisionnel et Orchestration des Flux Métiers)** : Le fichier `CommandeController.php` constitue l'axe central de la logique applicative du Back-End. Ce contrôleur REST intègre et applique l'ensemble des règles d'affaires (Business Rules), des verrous transactionnels et des contraintes juridiques qui régissent le cycle de vie d'une commande au sein de la centrale logistique. Il fait office de passerelle d'interconnexion entre les requêtes du Front-End, les flux d'importation externes et la persistance des données sous MySQL via l'ORM Eloquent.

Les processus critiques managés par ce composant s'articulent autour de quatre piliers techniques :

- **Moteur de Recherche Filtré et Indexation Dynamique (index)** : La méthode `index` prend en charge l'extraction segmentée des dossiers logistiques. Elle bâtit une requête SQL dynamique en interceptant les paramètres de filtrage optionnels (`statut_livraison` et

statut_entrepot). Par défaut, le moteur applique une clause d'exclusion stricte pour écarter les commandes finalisées (statut_livraison != 'livrée') afin d'alléger la charge réseau, tout en effectuant un chargement gourmand (*Eager Loading*) de la relation client pour éviter le problème de requête N+1.

- **Passerelle d'Interopérabilité ERP (importerDepuisErp)** : Ce point d'entrée expose l'API de l'application aux systèmes tiers, spécifiquement l'ERP Microsoft Dynamics AX. Il sécurise l'ingestion des données à l'aide de la façade Validator, qui valide l'unicité de la transaction, l'existence du profil client associé, la conformité de l'horodatage de paiement et la cohérence de la valeur marchande (montant_ttc), garantissant l'intégrité du patrimoine informationnel dès son acquisition.
- **Machine d'États et Verrous Réglementaires (planifierLivraison, annulerCommande, basculerForceMajeure)** : Ces méthodes sécurisent les transitions d'états des dossiers. Le système intercepte et rejette (Code HTTP 403) toute tentative de planification sur un dossier bloqué en surstock (post_delai) ou déjà clôturé. De même, la procédure de rupture contractuelle (annulerCommande) valide de manière algorithmique que le retard de livraison imputable à l'enseigne dépasse le seuil strict de 7 jours au moyen de la bibliothèque Carbon. En cas de validation, elle bascule automatiquement le dossier sous le statut de remboursement en_attente et calcule une date limite de restitution des fonds fixée à J+15.
- **Génération d'Actes Juridiques et Flux Binaires (telechargerMiseEnDemeure)** : Ce module automatise la production de pièces justificatives réglementaires. Après avoir validé que la marchandise a officiellement atteint le stade d'infraction contractuelle requis, le contrôleur exploite la bibliothèque DomPDF pour compiler dynamiquement les métadonnées du dossier au sein d'un gabarit HTML/CSS (pdf.mise_en_demeure). Le document est mis en page au format universel A4 Portrait, puis converti en un flux binaire brut (Blob) encapsulé dans une réponse HTTP de téléchargement forcé

4. **DashboardController.php (Moteur d'Agrégation Analytique et Piste d'Audit Comptable)** : Le fichier DashboardController.php fait office de centrale d'agrégation de données et de moteur de reporting décisionnel en temps réel pour l'application. Sa responsabilité exclusive est de compiler, calculer et structurer sous forme d'indicateurs de performance (KPIs) l'ensemble des métriques d'activité logistique, ainsi que de restituer les registres d'audit légaux de la plateforme. Il fournit au Front-End React les données brutes nécessaires à l'hydratation de la console de supervision.

L'architecture des traitements internes de ce composant repose sur trois axes logiques fondamentaux :

- **Calcul d'Indicateurs Métiers et Isolation des Cycles Légaux (getStats)** : La méthode getStats interroge de manière simultanée la base de données MySQL pour segmenter le stock de marchandises selon le cycle de vie légal et contractuel défini par l'enseigne. À l'aide de requêtes de comptage (count()), elle ventile les volumes de colis selon trois stades d'incrémentatation : la phase d'entreposage initial gratuit, la phase d'alerte intermédiaire (notifiée à J+30), et le stade contentieux (mise_en_demeure à J+60). Parallèlement, elle utilise des fonctions d'agrégation SQL (sum('montant')) pour évaluer instantanément la valeur cumulée des pénalités logistiques actives à l'échelle du réseau.
- **Algorithme d'Évaluation de la Responsabilité Transport (delayedShipments)** : Ce sous-système isole les défaillances de distribution imputables à la supply chain interne de KITEA. La fonction intercepte les commandes dont la livraison est planifiée, mais dont l'échéance prévisionnelle (date_livraison_prevue) est dépassée. À travers une fonction de filtrage de collection adossée à l'enveloppe temporelle Carbon, elle isole de manière

algorithmique les dossiers présentant une anomalie de retard supérieure à 7 jours révolus, quantifiant ainsi les cas ouvrant droit à une annulation client légitime.

- **Restitution des Pistes d'Audit Logistiques et Financières (historiqueAlertes, getPenaltyLogs) :** Les points d'entrée transactionnels dédiés à l'historique et au grand livre des frais permettent de maintenir une traçabilité immuable des événements du système. La méthode historiqueAlertes extrait la collection des logs de notifications classée par ordre antichronologique. De son côté, la méthode getPenaltyLogs matérialise le registre comptable des infractions en préchargeant via l'ORM Eloquent l'arbre de relations imbriquées commande.client (*Nested Eager Loading*). Ce mécanisme garantit qu'aucune pénalité ne peut être affichée sans sa justification d'origine, sécurisant ainsi le contrôle et la transparence des litiges de la supply chain.

5. **AuthController.php (Passerelle de Sécurité, Contrôle d'Accès et Gestion de Sessions) :** Le fichier AuthController.php centralise la gestion de la sécurité périmétrique, le provisionnement des comptes et le cycle de vie des sessions au sein de la plateforme logistique. S'appuyant sur le système d'authentification par jetons d'état déconnectés de Laravel Sanctum, ce contrôleur REST est chargé de certifier l'identité des opérateurs de la supply chain avant de leur octroyer des privilèges d'accès sur l'API, garantissant ainsi l'étanchéité des ressources contre les accès non autorisés.

L'architecture de sécurité de ce module est gouvernée par trois fonctions asynchrones principales :

- **Vérification d'Identité et Émission de Jetons Cryptographiques (login) :** La méthode login prend en charge la cinématique d'authentification des utilisateurs. Elle commence par valider la structure des données transmises (email et mot_de_pass) à l'aide de filtres de types stricts. Elle interroge ensuite la base de données MySQL pour extraire le profil utilisateur correspondant. L'évaluation de la légitimité des identifiants s'opère au moyen de la façade Hash::check(), qui compare en temps constant le mot de passe en clair soumis au hash cryptographique (généralement Bcrypt ou Argon2id) stocké en base de données. Si le contrôle est validé, le système invoque la méthode createToken() pour générer un jeton de sécurité Bearer unique au format opaque (plainTextToken), renvoyé au client parallèlement au profil d'habilitation (role) de l'agent.
- **Provisionnement des Profils Métiers et Règles d'Inclusion (register) :** Dédiée à la création de nouveaux comptes et protégée au sein du groupe de routes sécurisées, la méthode register assure l'inscription de nouveaux profils opérateurs. Elle embarque un validateur robuste qui garantit l'unicité stricte de l'adresse e-mail au sein du système. Elle intègre de surcroît une règle d'inclusion stricte (Rule::in) qui restreint impérativement l'attribution des rôles applicatifs aux seules valeurs métier autorisées (agent ou administrateur), interdisant toute tentative d'élévation de privilèges ou d'injection de statuts invalides. Avant la persistance dans la base de données via l'ORM Eloquent, le mot de passe subit un hachage irréversible via Hash::make().
- **Révocation de Session et Répudiation des Jetons (logout) :** Enfin, la méthode logout orchestre la fermeture sécurisée de la session de l'opérateur. En interceptant l'instance de l'utilisateur authentifié rattachée à la requête, elle isole le jeton d'accès courant (currentAccessToken()) et exécute sa suppression définitive au sein du registre de la base de données. Ce processus invalide instantanément le jeton Bearer côté serveur, neutralisant toute tentative de réutilisation ultérieure (attaque par rejeu) et fermant de fait la session logistique de manière déterministe.

6. **Couche de Persistance et Mappage Objet-Relationnel (Modèles Eloquent) :** La persistance, l'intégrité et la sémantique relationnelle des données logistiques et financières sont entièrement régies par la couche des modèles de l'ORM Eloquent de Laravel. Ces classes encapsulent la logique métier sous forme d'objets manipulables tout en abstrayant la complexité des requêtes SQL natives vers le serveur MySQL. Elles définissent les clés primaires personnalisées (\$primaryKey), les politiques d'assignation de masse (\$fillable) ainsi que les cascades de dépendances de l'application.

Cette infrastructure de persistance s'articule autour des entités et associations suivantes :

- **Modèle Pivot Commande.php :** Ce modèle centralise l'ensemble des métadonnées opérationnelles d'une transaction. Il redéfinit sa clé primaire sur l'attribut `id_commande` pour s'aligner sur les structures de données importées de l'ERP AX. À travers des relations typées `HasMany`, il orchestre la traçabilité descendante des infractions contractuelles vers le registre des notifications (Notification) et le grand livre des charges financières (Frais). Inversement, ses liaisons ascendantes `BelongsTo` lui permettent de rattacher chaque colis à un acheteur unique (Client) ainsi qu'à l'agent de transit responsable de sa planification (User).
 - **Gestion des Profils Tiers et Opérateurs (Client.php et User.php) :** Le modèle Client encapsule l'identité civile et électronique des acheteurs nécessaires aux serveurs SMTP pour l'expédition des relances à J+30 et J+60. Le modèle User, de son côté, hérite de la classe `Authenticatable` pour supporter l'hydratation des contextes de sécurité. Il implémente le trait `HasApiTokens` propre à Laravel Sanctum pour gérer l'émission des clés de session déconnectées, et utilise le tableau `$hidden` pour interdire toute exposition de l'empreinte du mot de passe (`mot_de_pass`) lors des flux d'échange avec l'interface React.
 - **Registres d'Audit de Contentieux (Frais.php et Notification.php) :** Ces entités matérialisent les tables d'historisation immuables du système. Le modèle Frais consigne de manière isolée chaque application de pénalité financière à J+60 en conservant le montant calculé et son horodatage d'effet. Le modèle Notification remplit une fonction similaire pour la traçabilité des communications, archivant le type de démarche administrative effectuée (relance de courtoisie ou mise en demeure officielle) ainsi que le statut de distribution du message, garantissant ainsi la validité légale des procédures d'entrepôt.
7. **Couche de Notification Métier et Flux de Communication Automatisés (Mailables) :** La gestion des interactions clients, le traitement des relances amiables et la notification formelle des infractions contractuelles sont délégués à la couche des sous-systèmes de messagerie du Back-End. Modélisées à l'aide des classes *Mailables* de Laravel, ces entités convertissent les changements d'état logistiques détectés par le système en vecteurs de communication normalisés. Elles se chargent d'encapsuler la configuration des protocoles d'envoi, de structurer l'arborescence des templates de courriels et de compiler les documents de preuve juridique requis.

L'architecture de cette couche repose sur deux classes spécialisées reflétant les fenêtres réglementaires de la centrale logistique :

- **Vecteur d'Alerte Amiable (RelanceStockage.php)** : Ce module prend en charge l'expédition de la notification de courtoisie déclenchée au seuil critique de J+30. La méthode `envelope()` injecte dynamiquement l'identifiant métier de la transaction (`id_commande`) dans l'objet du message afin d'optimiser le taux d'ouverture et de faciliter le traitement côté client. Elle instancie un objet `Content` pointant vers la vue HTML/CSS isolée `emails.relance_stockage`, qui formule un rappel des conditions de rétention physique des colis sans y joindre de charge financière, la méthode `attachments()` restant volontairement vide.
 - **Vecteur Contentieux et Compilation de Flux à la Volée (MiseEnDemeureStockage.php)** : Intervenant lors du franchissement du cap réglementaire des 60 jours, cette entité gère la mise en demeure formelle. Outre l'hydratation de l'enveloppe et du corps de l'e-mail (`emails.mise_en_demeur`), sa plus-value réside dans son moteur d'association de pièces jointes. La méthode `attachments()` intercepte à l'exécution le modèle de commande et sollicite la façade `DomPDF` pour compiler en mémoire RAM le modèle d'acte officiel au format A4. Grâce à la méthode `Attachment::fromData()`, le flux binaire généré par `pdf->output()` est injecté directement dans le canal de transport SMTP sous le type MIME `application/pdf`, éliminant le besoin d'écrire ou de stocker temporairement le document sur le serveur physique, garantissant ainsi une sécurité de données optimale.
8. **console.php / Kernel.php (Orchestration Temporelle et Planification des Tâches)** : Le fichier de configuration de la console centralise la déclaration des routines automatisées et l'orchestration temporelle (Task Scheduling) de l'application. S'appuyant sur le gestionnaire de tâches natif de Laravel, ce script élimine la dépendance complexe à de multiples configurations de tâches Cron système au niveau du serveur hôte en centralisant la table des fréquences d'exécution directement dans le code source de l'application.

L'intégration du script d'arrière-plan repose sur les mécanismes suivants :

- **Enregistrement des Commandes de Console** : Le fichier fait office de registre d'exécution en associant des signatures textuelles personnalisées (telles que `app:check-storage-delays`) à leurs classes ou fermetures (*closures*) de traitement respectives. Ce mapping permet au système d'invoquer les traitements métiers complexes directement depuis l'interface en ligne de commande (CLI) de Laravel Artisan.
- **Régulation des Fréquences d'Exécution et Cycle Automone** : À travers l'architecture de la façade `Schedule`, le système définit de manière déterministe la récurrence de l'audit des stocks. Configuré sous la directive `everyMinute()` lors des phases de tests unitaires et de validation fonctionnelle pour simuler l'obsolescence accélérée des marchandises, le planificateur est nativement conçu pour être basculé sous la directive `daily()` en environnement de production. Cette planification quotidienne déclenche le cycle autonome de vérification de vétusté, d'application des pénalités financières et d'expédition des actes de mise en demeure chaque nuit à heure fixe, garantissant une autonomie totale et une synchronisation rigoureuse de la plateforme logistique sans intervention humaine.

4.3 Fonctionnalités principales

L'application offre un écosystème centralisé de fonctionnalités permettant d'automatiser le cycle de vie contractuel des marchandises, de sécuriser les flux financiers liés aux pénalités et d'orchestrer les livraisons de manière fluide et traçable. Voici un aperçu détaillé des principales fonctionnalités :

- **Authentification sécurisée et gestion des profils métiers** : L'application implémente un contrôle d'accès strict basé sur les rôles (RBAC). Les opérateurs de la supply chain s'authentifient de manière sécurisée via des jetons d'état (Bearer Tokens) gérés par le Back-End. Les administrateurs possèdent des privilèges avancés leur permettant de provisionner de nouveaux comptes utilisateurs et de restreindre les droits d'accès aux fonctionnalités critiques (telles que le gel des pénalités ou la validation des annulations) selon que l'utilisateur possède le rôle d'agent ou de superviseur.
- **Passerelle d'ingestion et d'importation automatique depuis l'ERP** : Le système expose une API REST dédiée à l'interopérabilité avec l'ERP Microsoft Dynamics AX de l'enseigne. Dès qu'une commande est acquittée financièrement côté ERP, ses métadonnées (identifiants, informations clients, horodatages de paiement et montants TTC) sont automatiquement ingérées et persistées dans l'application. Cette fonctionnalité garantit l'alignement en temps réel du patrimoine d'informations entre les services comptables et la centrale logistique, marquant le point de départ du suivi d'entrepôt.
- **Planification et contrôle des fenêtres de distribution** : L'application met à la disposition des agents un module de planification logistique. Pour chaque commande importée, l'opérateur peut affecter une date prévisionnelle de livraison. Le système intègre des verrous métiers stricts empêchant la planification ou la modification de dossiers dont le délai de gratuité est dépassé, ou ayant fait l'objet d'une rupture contractuelle préalable, sécurisant ainsi l'agenda des expéditions.
- **Moteur d'audit de vétusté et liquidation automatique des frais (J+30 / J+60)** : Un service d'arrière-plan autonome (Cron Job) audite chaque nuit l'ensemble des marchandises stockées à l'entrepôt. À J+30 de rétention physique, le système modifie automatiquement le statut du dossier pour engager une phase d'alerte. Au franchissement du seuil légal des 60 jours, le moteur applique de manière autonome une pénalité financière stricte de 10 % de la valeur TTC de la commande, consigne la transaction dans le grand livre des frais logistiques et génère l'acte juridique requis.
- **Arbitrage des cas de Force Majeure et gestion des contentieux (Post-Délai)** : L'application intègre un mécanisme de traitement des exceptions contractuelles. En cas d'événement imprévisible et extérieur, un commutateur permet de basculer un dossier sous le régime de la "Force Majeure", gelant instantanément le calcul des pénalités et bloquant les droits d'annulation. Inversement, si un retard de distribution supérieur à 7 jours est imputable à KITEA, le système qualifie le dossier en "post_delai", levant une alerte critique à l'écran et ouvrant le droit au client de solliciter l'annulation de sa commande et l'émission d'un remboursement sous un délai légal de 15 jours.
- **Système de notification omnicanal et génération d'actes juridiques** : Afin de garantir une parfaite transparence et la validité juridique des démarches de l'entrepôt, l'application est dotée d'un sous-système de notification automatisé. Lors des bascules d'état critiques (J+30 et J+60), le Back-End expédie instantanément des e-mails de relance formalisés aux acheteurs. Pour le stade contentieux de la mise en demeure (J+60), le système compile à la volée un document officiel au format PDF A4 Portrait reprenant l'historique et les frais appliqués, envoyé en pièce jointe et disponible en téléchargement pour l'agent.

